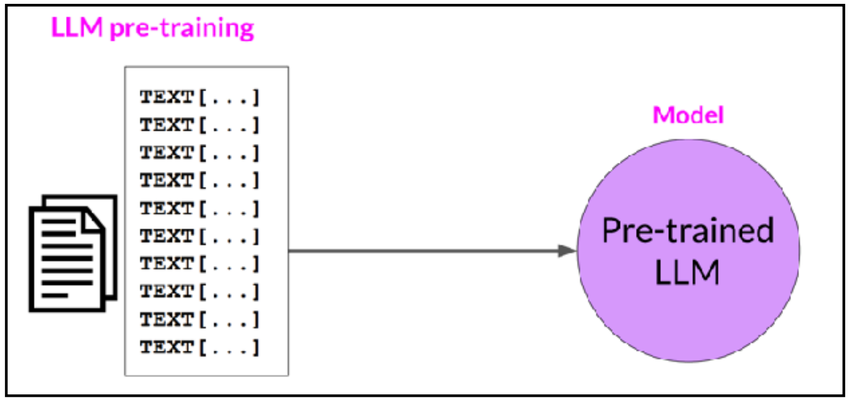
Understanding Large Language Models

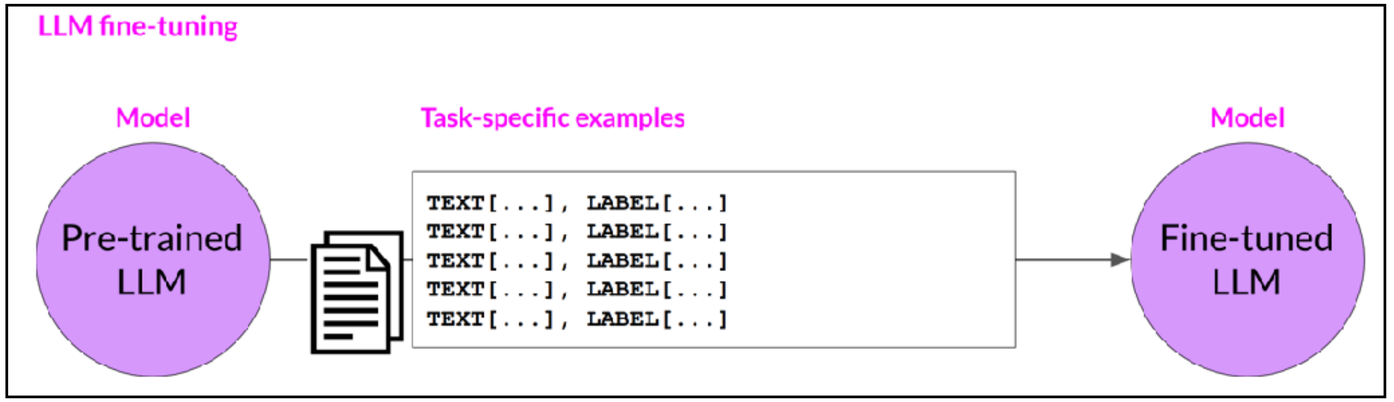
Carsten Eickhoff, Michael Franke and Polina Tsvilodub

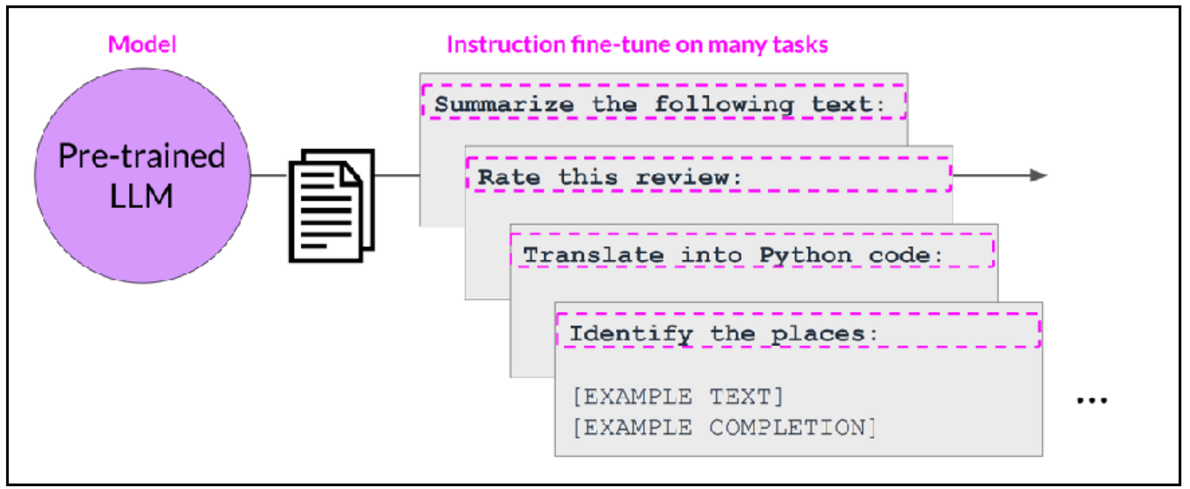
Session 06: in-context learning, tool use, LM applications, agents

Recap & roadmap

Pre-training & (instruction) fine-tuning

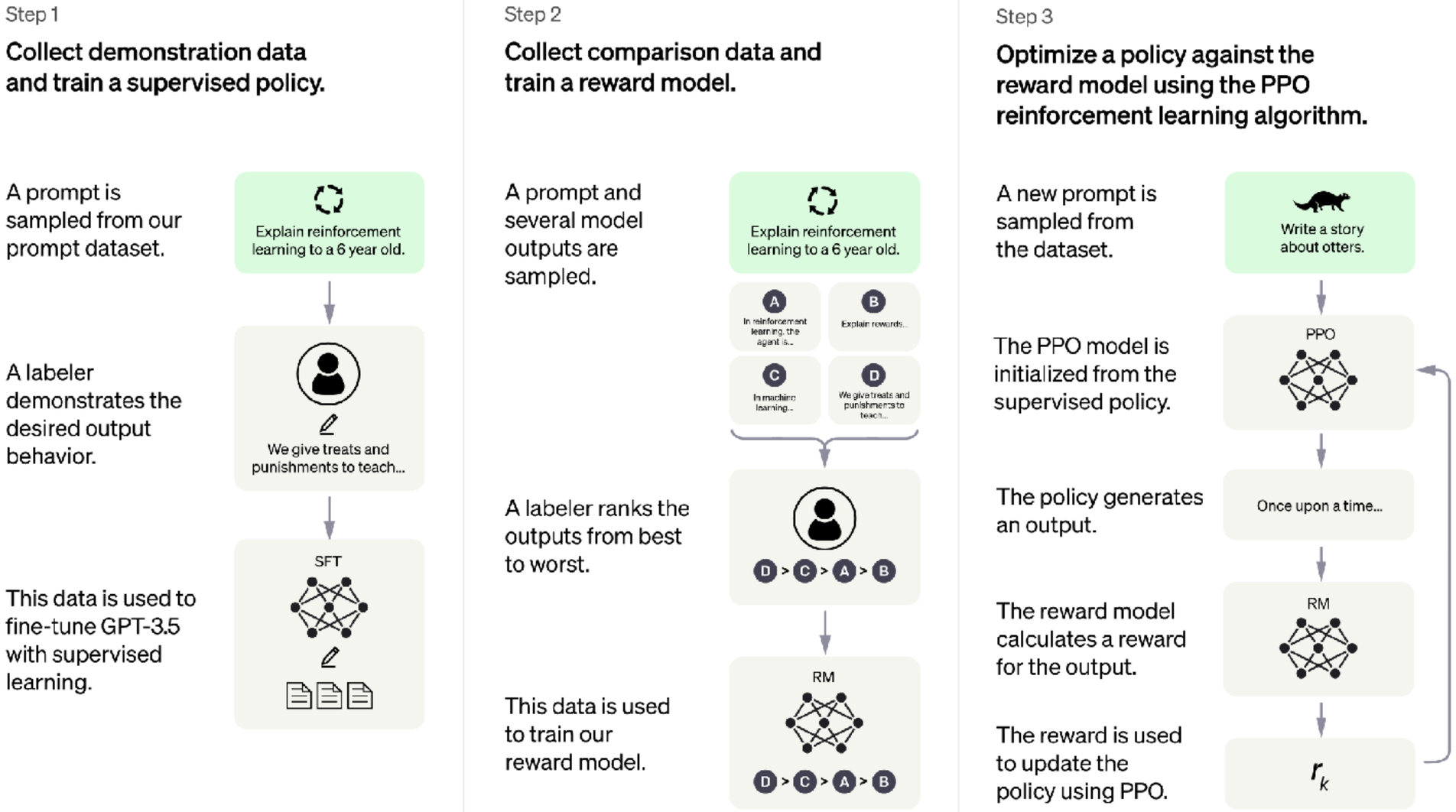




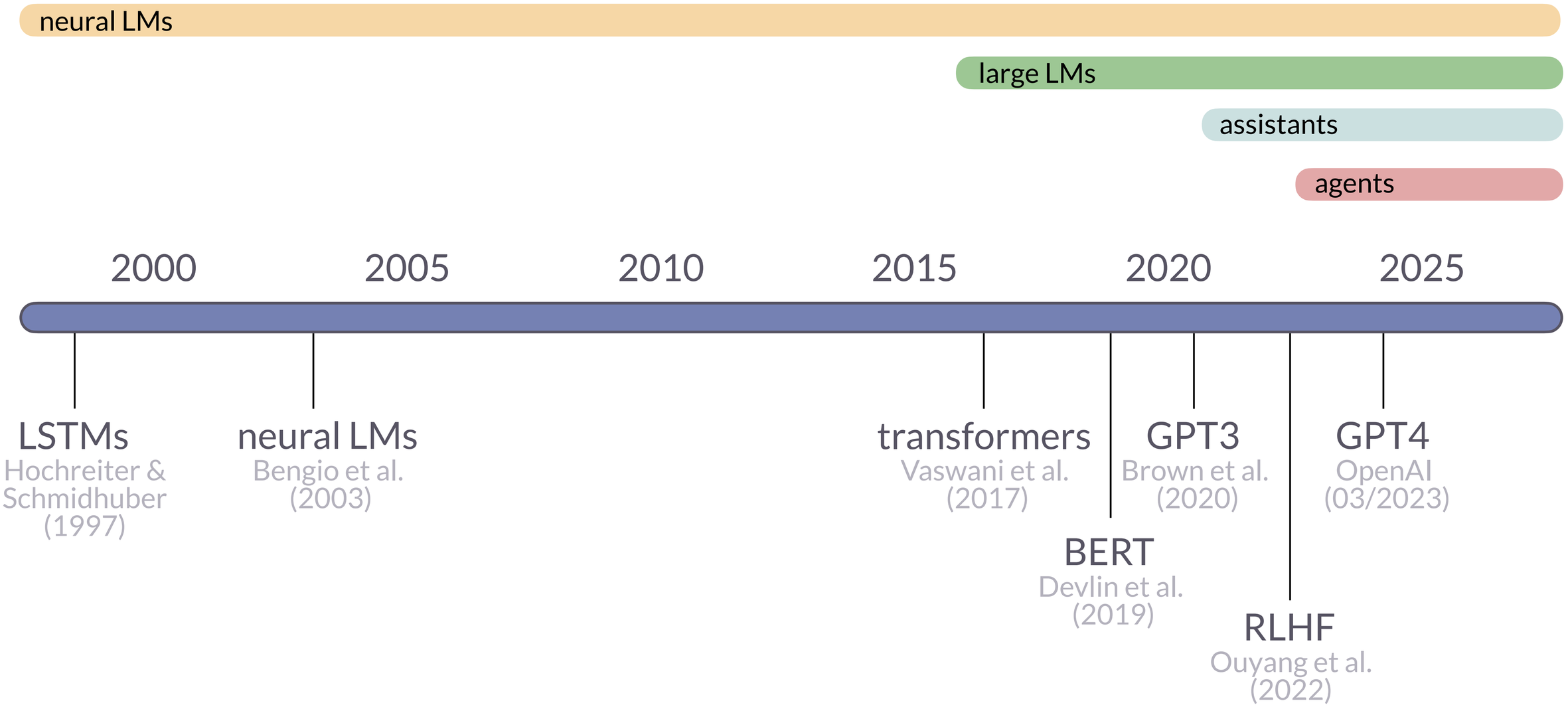


Human feedback in RL

RLHF



OpenAI (2022)


neural LMs
large LMs
assistants
agents
2000
2005
2010
2015
2020
2025
LSTMs
Hochreiter & Schmidhuber (1997)
neural LMs
Bengio et al. (2003)
transformers
Vaswani et al. (2017)
BERT
Devlin et al. (2019)
GPT3
Brown et al. (2020)
RLHF
Ouyang et al. (2022)
GPT4
OpenAI (03/2023)

Kinds of Large Language Models

core LLMs
(foundation models)

- predict statistically likely next token
- e.g.,
  - GPT-2
  - LLaMA2 / LLaMA3
  - …

earlier

prepped LLMs
(assistants)

- fine-tuned (e.g. RLHF)
- predict token likely to please the user
- e.g.,
  - GPT-3.5
  - LLaMA3 Instruct
  - …

last session & today

LLM-based applications
(agents)

- algorithm using LLMs
- e.g.:
  - sophisticated prompting
  - Chat-GPT w/ tools
  - AutoGPT
  - neuro-symbolic models
  - …

today & later

LLM Assistants from RLHF

non-toxic, well-structured & helpful

Recap

- pre-trained core / base / **foundation model**
  - language modeling objective: predict next word, which is statistically likely given your training data

"Here is a fragment of text …
According to your **knowledge of the statistics of human language**, what words are likely to come next?

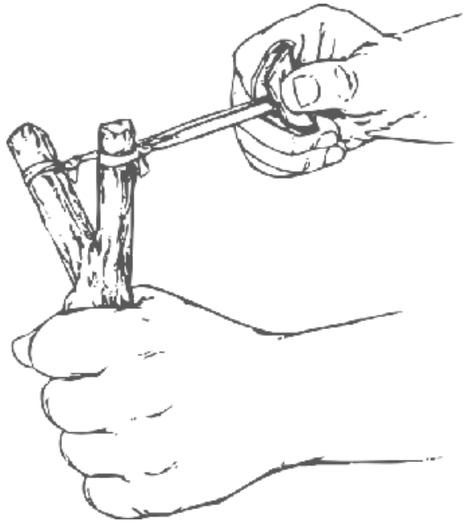
Shanahan (2022)

- prepped **assistants**
  - usefulness objective: produce text that satisfies user goals

"Here is a fragment of text …
According to your **reward-based conditioning**, what words are likely to trigger positive feedback?"

caveat: this can feel like the LLM "understands" your request / intention

Main learning goals

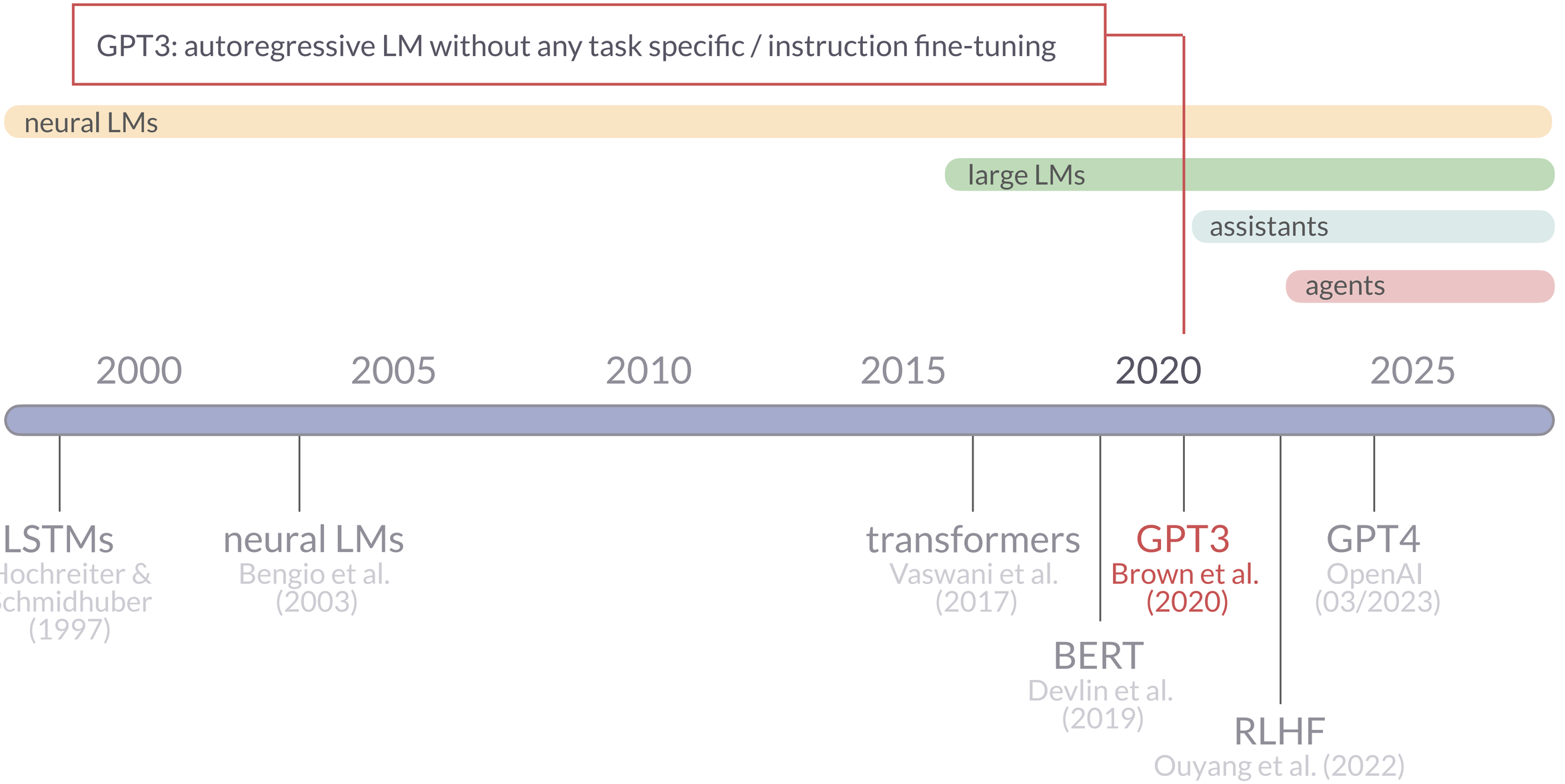


1. In-context learning
   - few-shot prompting, properties, explanations
2. Prompt engineering
   - Chain of thought, generated knowledge prompting, tree of thoughts
3. LM-based applications
   - retrieval augmented generation, chat agents, tool users
   - autonomous agents, simulacra
   - LMs in robotics & RL

In-Context Learning

“Language Models are Few-Shot Learners”

Brown et al. (2020)



Zero-shot prompting

- may include task instruction, but has no example or illustration
  - works well only for models fine-tuned on instruction-following data
  - works well only for frequent (simple) tasks

INPUT

```
Classify the sentence as positive, neutral or negative.
Sentence: This class is super exciting!
Sentiment:
```

OUTPUT

```
positive
```

In-context learning

a.k.a. k-shot learning

- prompt with k pairs of **demonstrations** (x_i, y_i)
  - x_i is the **input**
  - y_i is the **output** or **label**
- add only the target input x_t of test item (x_t, y_t)
- ICL is an emerging ability
  - boosts performance on common tasks, even though models are NOT trained do in-context learning
  - shown first for GPT-3 by Brown et al. (2020)
- works with or without (!) task instructions
- common observation:
  - performance increases with size k of demonstrations

INPUT

```
This class in interesting. neutral
This class is my favorite. positive
This class is a drag. negative
This class is super exciting.
```

OUTPUT

```
positive
```

ICL in perspective

- ICL may not be “learning” proper
  - Reynolds & McDonell (2021)
- conditions influencing ICL performance
  - Min et al. (2022)
- ICL without “prompt understanding”
  - Webson & Pavlick (2022)
- ICL improves with additional explanations
  - Lampinen et al. (2022)

Towards “prompt engineering”

Reynolds & McDonell (2021)

- contra claim “k-shot learning = meta-learning” *sensu strict*
- result: effect of examples can be retrieved by proper prompt
- interpretation: task ability comes from pre-training; prompt guides the system towards the “task location”

simple colon prompt

```
French: example_source_phrase
English: example_target_phrase

French: example_source_phrase
English: example_target_phrase

[...]

French: source_phrase
English:
```

master translator prompt

```
A French phrase is provided: source_phrase
The masterful French translator flawlessly
translates the phrase into English:
```

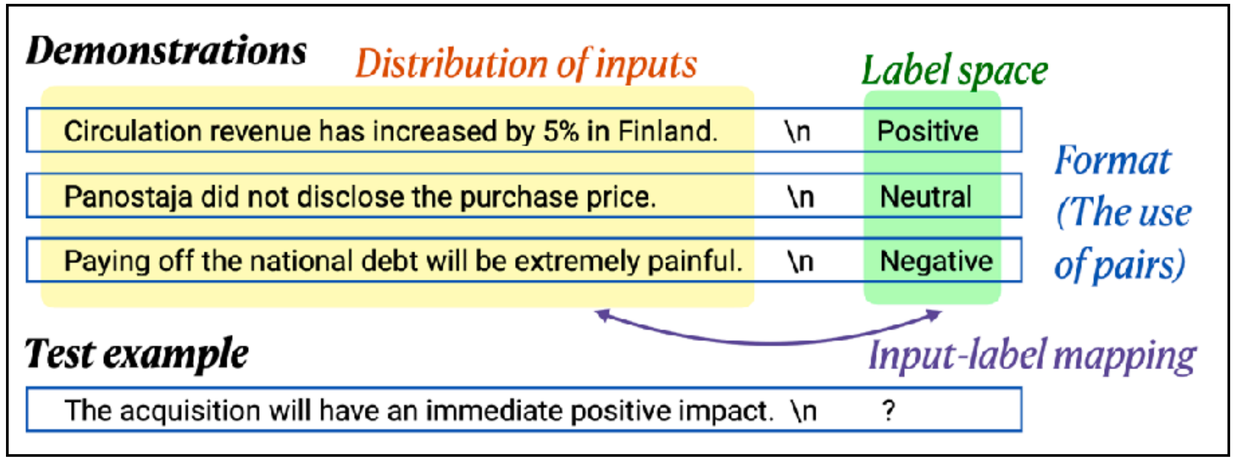
results

Prompt	Babbage / 6.7B	Curie / 13B
OpenAI 0-shot	15.5	22.4
OpenAI 1-shot	31.6	31.4
OpenAI 64-shot	36.4	38.3
Reproduced OpenAI 0-shot	15.9	18.7
Reproduced OpenAI 1-shot	21.8	24.1
Reproduced OpenAI 10-shot	25.1	27.9
Simple colon 0-shot	23.5	33.3
Simple colon 1-shot	18.0	27.6
Simple colon 10-shot	24.1	33.4
Master translator 0-shot	26.5	32.9

What makes in-context learning work?

Min et al. (2022)

- empirically test effect of four factors:
  - **correctness** of input-label mapping
  - **similarity** of examples and target input
  - **coverage** of label space in examples
  - **format** of input-label examples
- main results:
  - input-label correctness matters little
  - other factors matter more
- similarity of examples and target input matters
  - **possible explanation:** the more similar the demonstrations are to the target input, the more the task resembles autoregressive text-generation; the task becomes "more similar" to what is seen in the demonstrations
  - unlike "meta-learning" *sensu stricto*



Do models understand the meaning of the prompts?

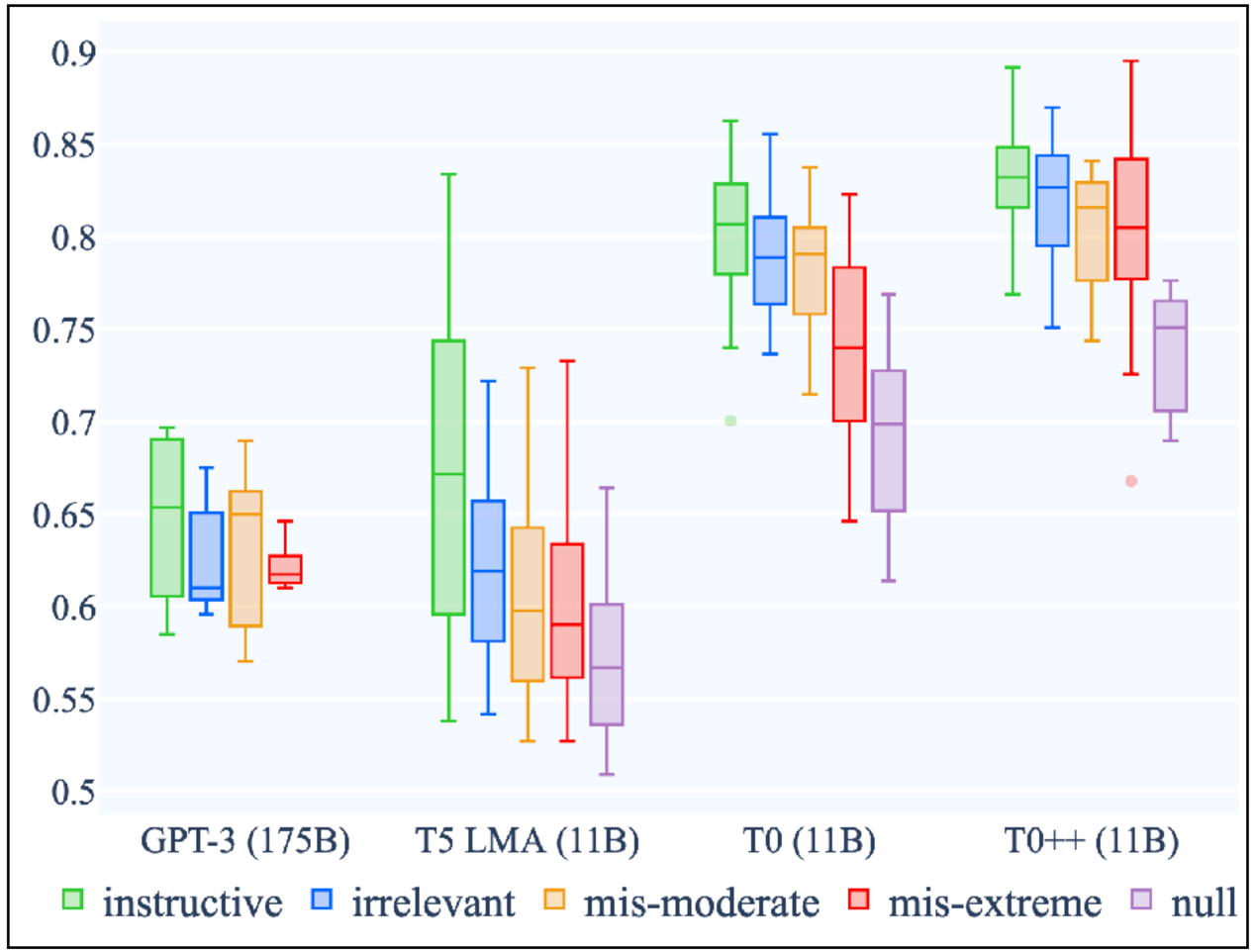
Webson & Pavlick (2022)

- zero-shot, and k-shot in-context learning
  - $k \in \{0,4,8,16,32,64,128,256\}$
- different example templates
  - see →
- different target words
  - yes/no
  - “yes/no”-like (true/false, positive/negative)
  - arbitrary (cat/dog, Jane/Jake)
  - reversed (no/yes)

Category	Examples
instructive	{prem} Are we justified in saying that “{hypo}”? Suppose {prem} Can we infer that “{hypo}”?
misleading-moderate	{prem} Can that be paraphrased as: “{hypo}”? {prem} Are there lots of similar words in “{hypo}”?
misleading-extreme	{prem} is the sentiment positive? {hypo} {prem} is this a sports news? {hypo}
irrelevant	{prem} If bonito flakes boil more than a few seconds the stock becomes too strong. "{hypo}"?
null	{premise} {hypothesis} {hypothesis} {premise}

Do models understand the meaning of the prompts?

Webson & Pavlick (2022)

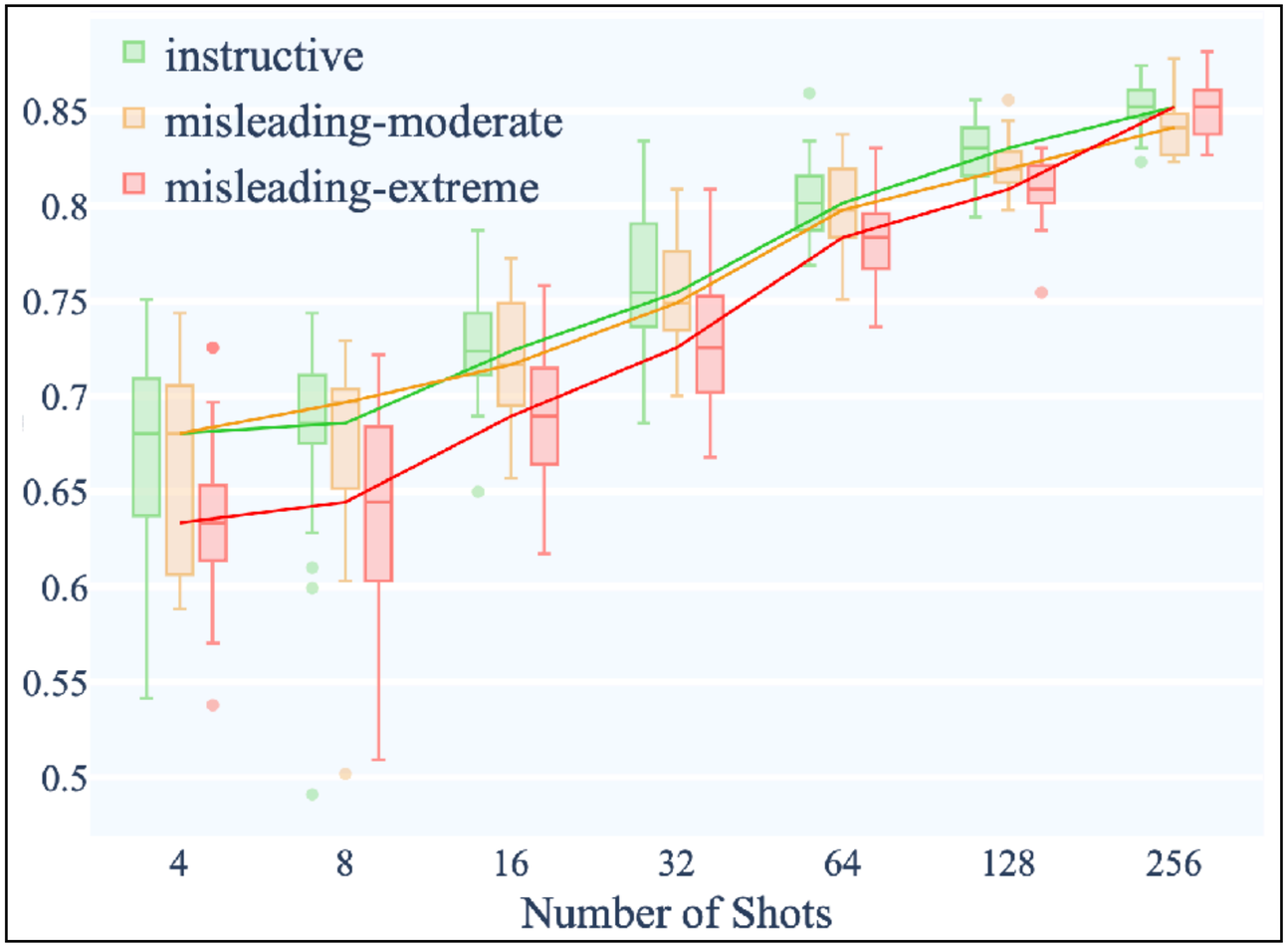


16-shot accuracy: no differences btw. categories except for comparisons against "null"

Category	Examples
instructive	{prem} Are we justified in saying that "{hypo}"? Suppose {prem} Can we infer that "{hypo}"?
misleading-moderate	{prem} Can that be paraphrased as: "{hypo}"? {prem} Are there lots of similar words in "{hypo}"?
misleading-extreme	{prem} is the sentiment positive? {hypo} {prem} is this a sports news? {hypo}
irrelevant	{prem} If bonito flakes boil more than a few seconds the stock becomes too strong. "{hypo}"?
null	{premise} {hypothesis} {hypothesis} {premise}

Do models understand the meaning of the prompts?

Webson & Pavlick (2022)



performance of T0 (3B): only difference btw. "misleading extreme" for 8 to 128 shots

Category	Examples
instructive	{prem} Are we justified in saying that "{hypo}"? Suppose {prem} Can we infer that "{hypo}"?
misleading-moderate	{prem} Can that be paraphrased as: "{hypo}"? {prem} Are there lots of similar words in "{hypo}"?
misleading-extreme	{prem} is the sentiment positive? {hypo} {prem} is this a sports news? {hypo}
irrelevant	{prem} If bonito flakes boil more than a few seconds the stock becomes too strong. "{hypo}"?
null	{premise} {hypothesis} {hypothesis} {premise}

Do models understand the meaning of the prompts?

Webson & Pavlick (2022)

- different target words
  - yes/no
  - “yes/no”-like (true/false, positive/negative)
  - arbitrary (cat/dog, Jane/Jake)
  - reversed (no/yes)
- target word matters for average success
- but unintuitive interactions with:
  - the former can outperform the latter
    - {premise} Does the paragraph start with “the"? {hypothesis} [yes/no]
    - {premise} Based on the previous passage, is it true that "{hypothesis}"? [cat/dog]

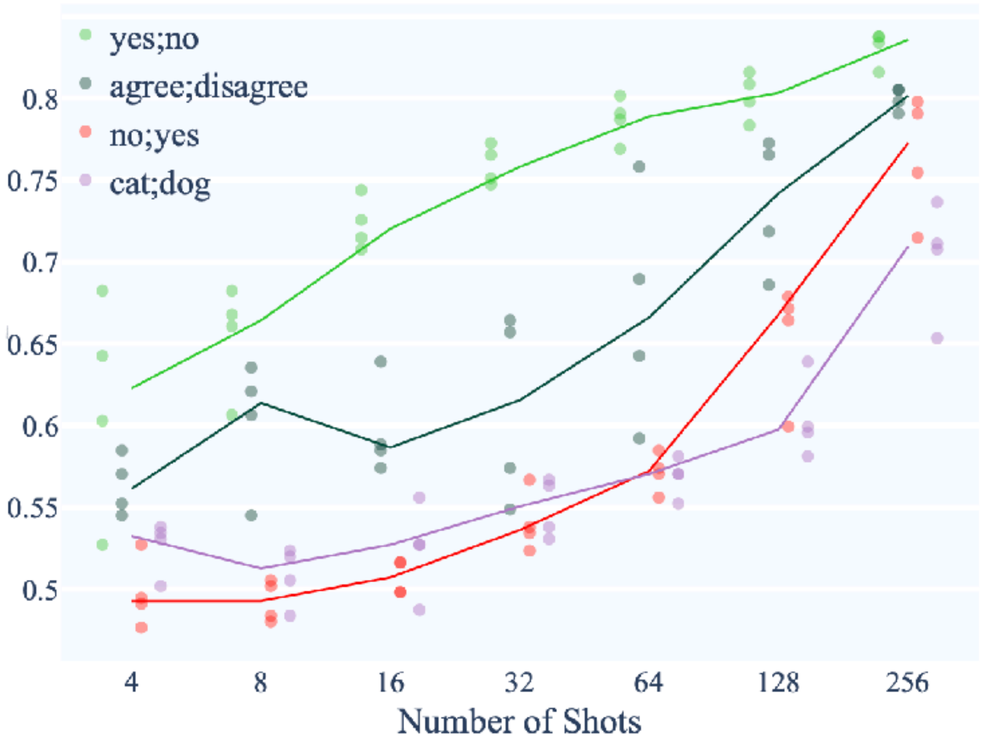
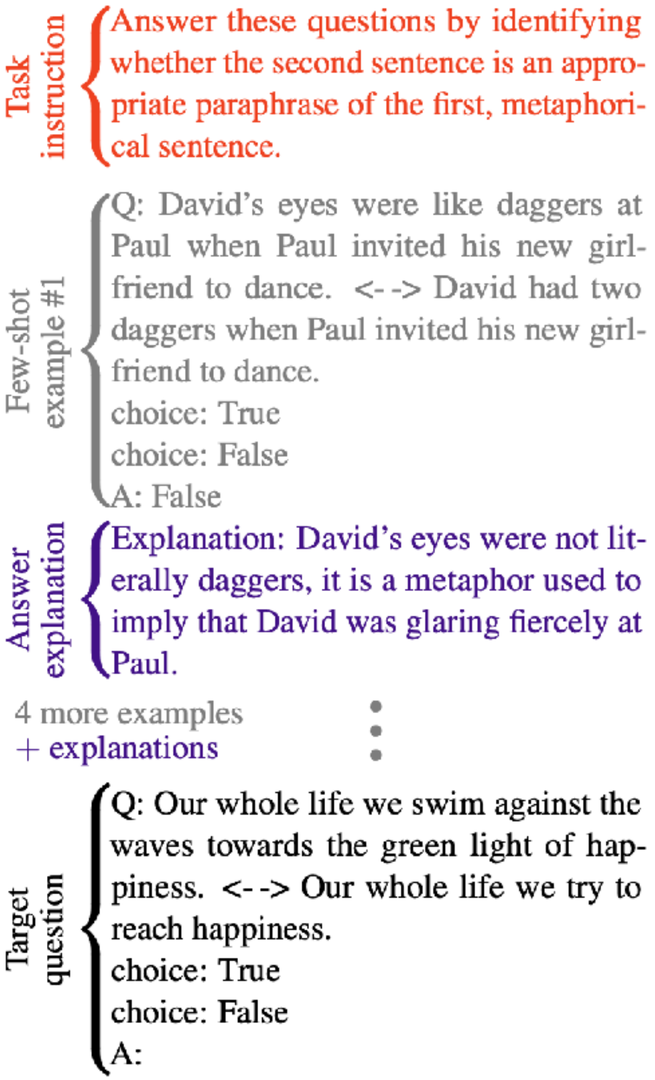


Figure 5: The best-performing instructive template for ALBERT on RTE, `{prem} Are we justified in saying that "{hypo}"?` with select LM targets from each category.

Can LMs learn from explanations in context?

Lampinen et al. (2022)

- method:
  - supplement ICL with additional examples
- main findings:
  - explanations **can** further improve ICL performance
  - hand-tuned explanations work better
  - benefit only for larger LMs



Summary
In-context learning

- structure of simple prompt also matters; so ICL need not be genuine “learning”
  - Reynolds & McDonell (2021)
- similarity of demonstrations and target matters
  - Min et al. (2022)
- instructions and label correctness do not matter
  - Min et al. (2022), Webson & Pavlick (2022)
- still, good explanations can help (for large LMs)
  - Lampinen et al. (2022)

Understanding ICL
up close

Understanding ICL

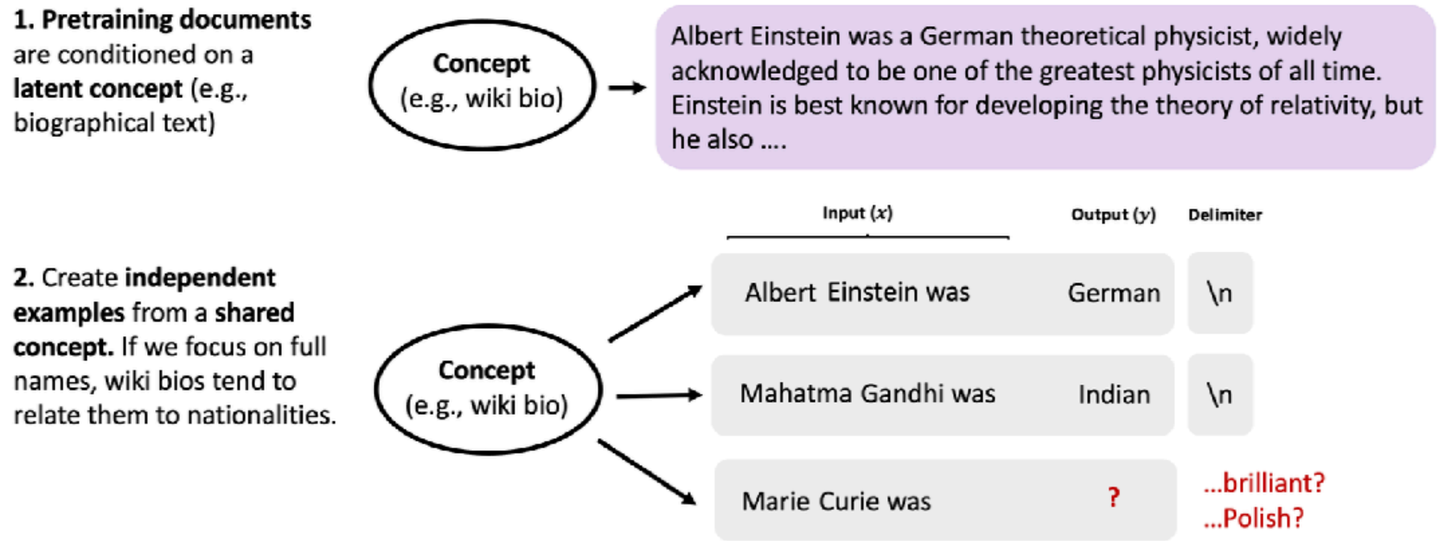
mentionables from an ongoing debate

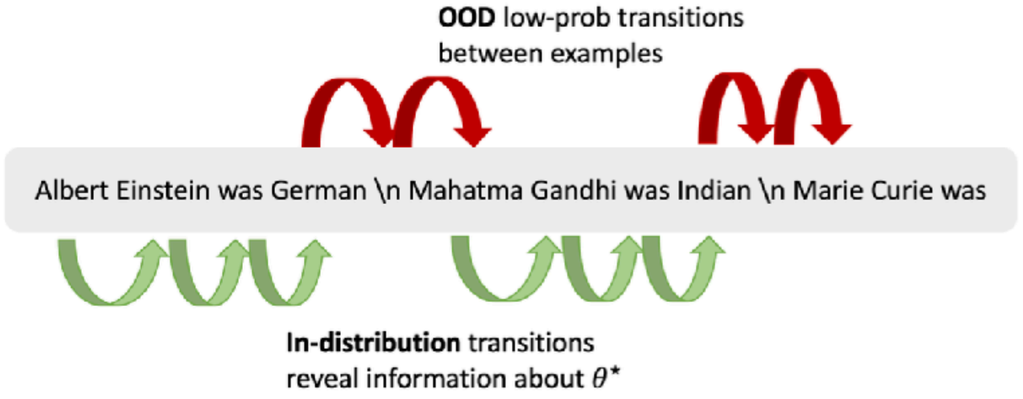
- ICL as implicit **Bayesian inference**
  - Xie et al. (2022)
- ICL can be recast as **function learning**, and may implement implicit **fine-tuning** (by GD)
  - Dai et al. (2022), Garg et al. (2022), Akyürek et al. (2023)
- ICL as implicit **structure induction**
  - Hahn & Goyal (2023)
- ICL relies on the emergence of **“induction heads”**
  - Olsson et al. (2022)
- ICL relies on the emergence of **“n-gram heads”**
  - Akyürek et al. (2024)
- maybe some of these analyses are a fair bit of **hot air**
  - Shen et al. (2024)

In-context learning as implicit Bayesian inference

Xie et al. (2022)

- starting point:
  - training data is generated by a **hierarchical** stochastic process
    - this is a more commonly expressed idea
- observation:
  - k-shot input is out-of-distribution
  - high within-example probability
  - low between-example probability
- conjecture:
  - in-context learning is inference of the **latent type** shared by demonstrations



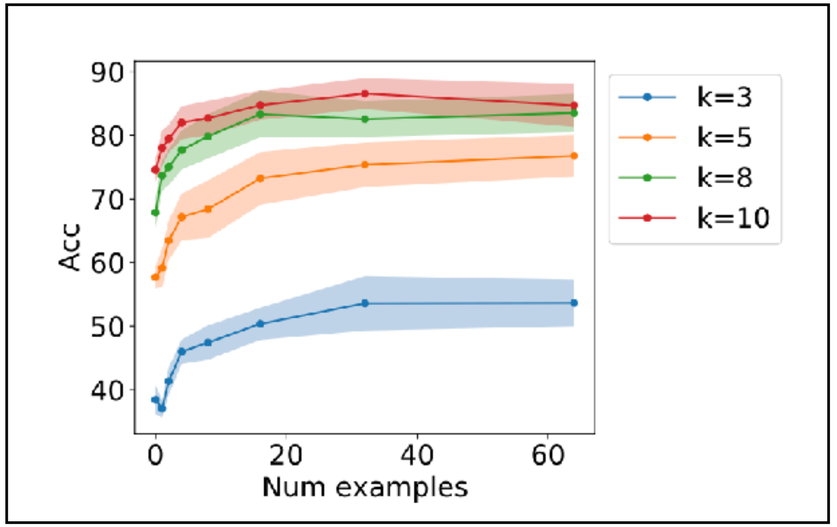


In-context learning as implicit Bayesian inference

Xie et al. (2022)

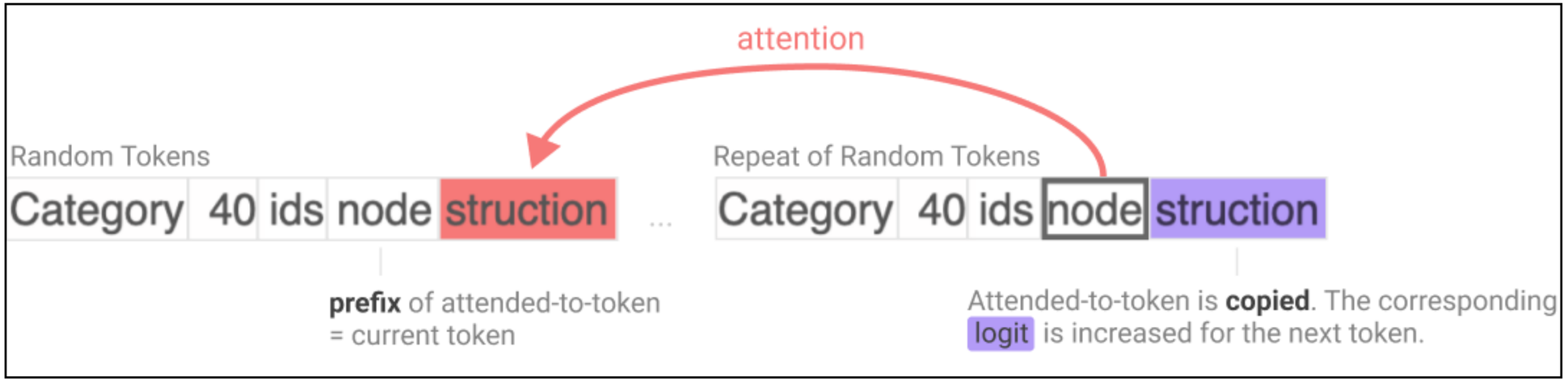
- theoretical argument:
  - proof that recovering the latent document type from k-shot examples is theoretically possible
- simulation results:
  - generate synthetic training data from a **hierarchical HMM process**
  - train autoregressive LSTM and transformer models (small-ish)
- results:
  - both LSTMs and transformers show **emerging ICL ability**
  - reproduces known effects:
    - more examples ⇒ better performance
    - example ordering matters
  - larger models ⇒ better ICL at equal predictive accuracy on train/test data

effect of examples numbers



In-context learning via induction heads

Olsson et al. (2022)



Hypothesis: "Induction heads might constitute the mechanism for the majority of all "in-context learning" in large transformer models."

SUMMARY OF EVIDENCE FOR SUB-CLAIMS (STRONGEST ARGUMENT FOR EACH)

	Small Attention-Only	Small with MLPs	Large Models
Contributes Some	Strong, Causal	Strong, Causal	Medium, Correlational & Mechanistic
Contributes Majority	Strong, Causal	Medium, Causal	Medium, Correlational

Terminology

- in-context learning
- few-shot learning
- structure-based example effects
- …?

How should we call it?

From simple to engineered prompting

Counting letters

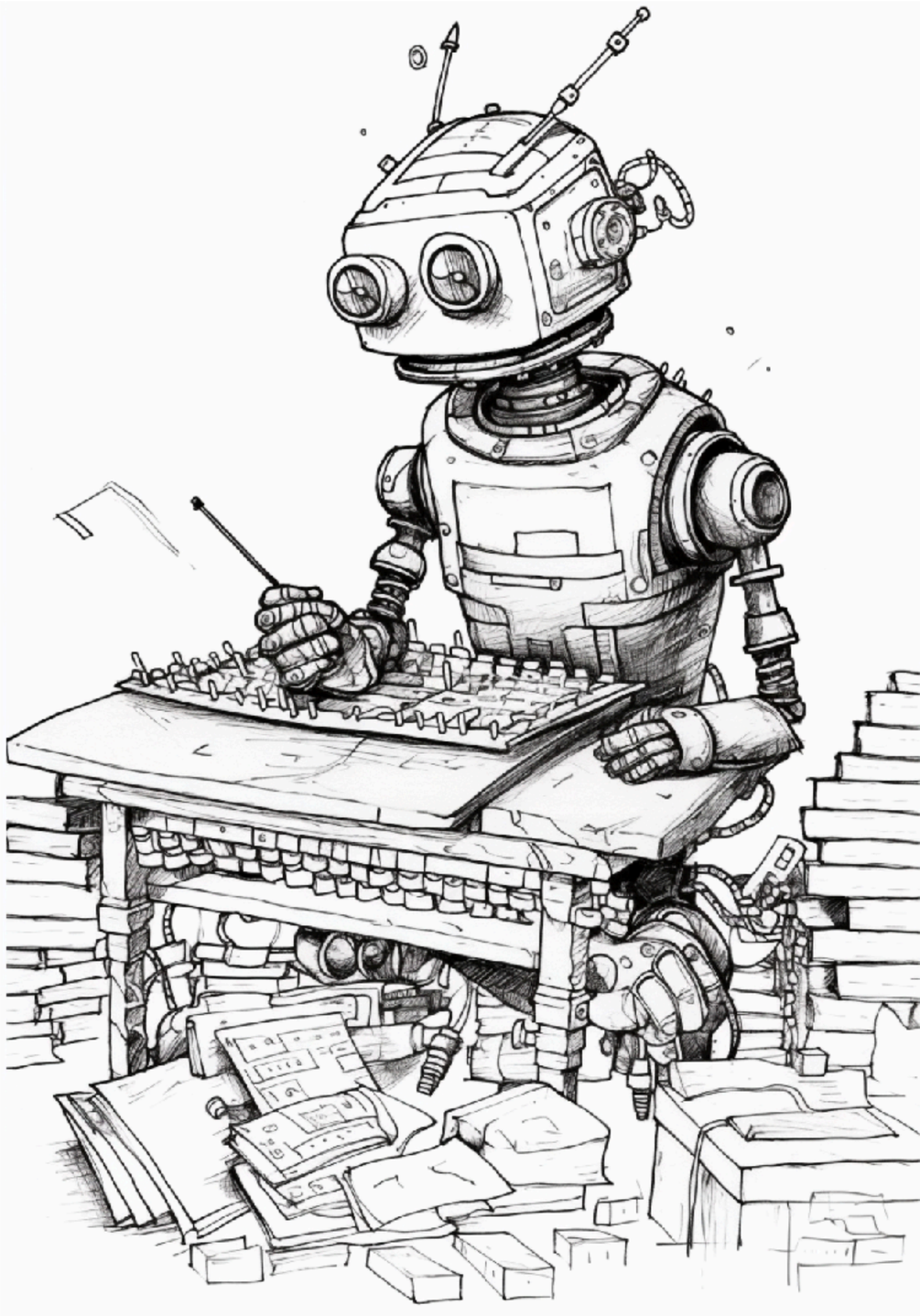
Exploring step-by-step reasoning

INPUT

```
Do the numbers of letters in all words starting with a
vowel from the following list sum up to 42?

Polina, Michael, eggplant, cheese, oyster, imagination,
elucidation, induce

Please answer just 'yes' or 'no'
```

OUTPUT

```
>>> ???
```



Structured reasoning

GPT-3.5 turbo (Dec 2023)

Do the numbers of letters in all words starting with a vowel from the following list sum up to 42?

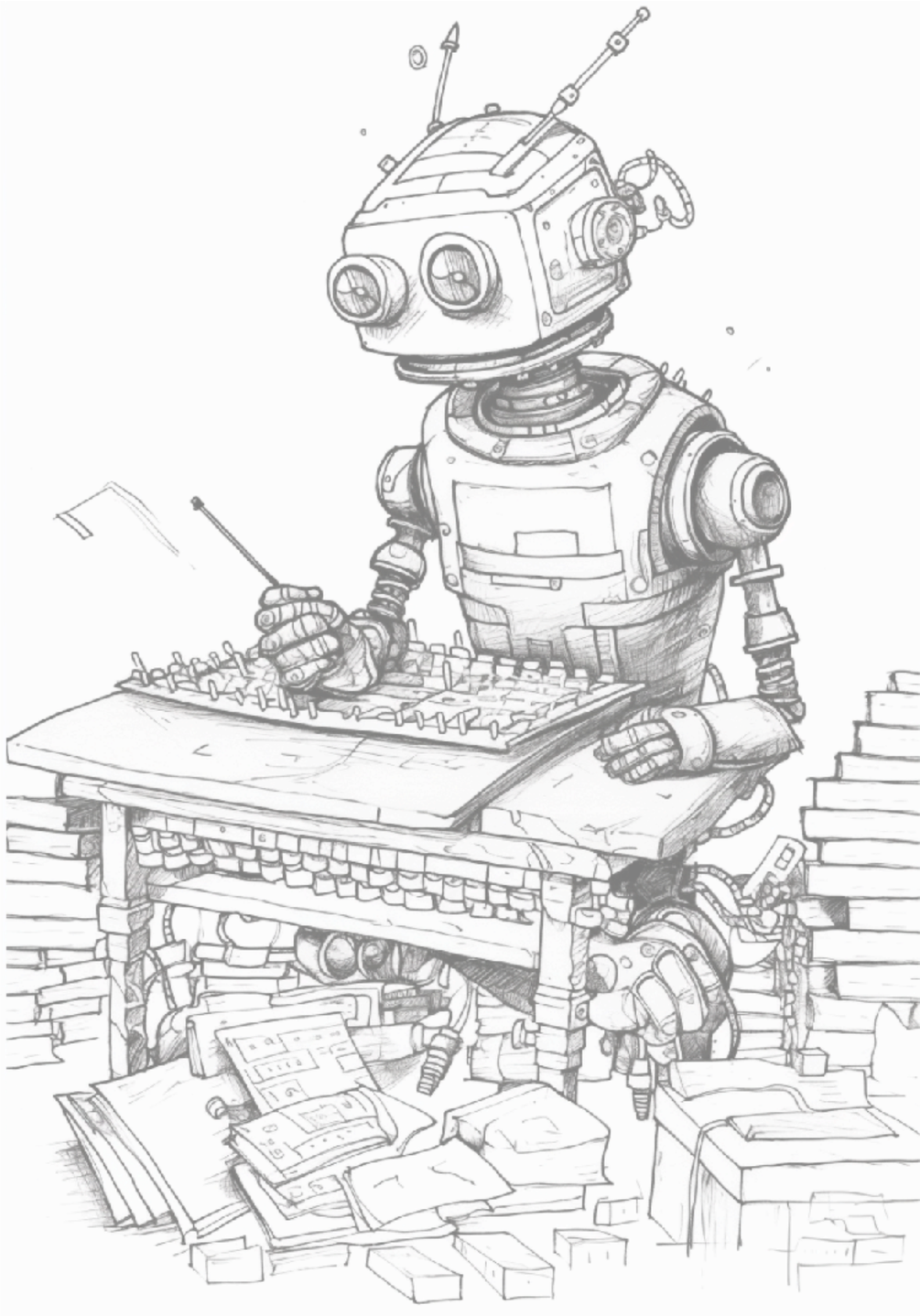
Polina, Michael, eggplant, cheese, oyster, imagination, elucidation, induce

Please answer just 'yes' or 'no'

No

No = 98.42%
Yes = 1.57%
NO = 0.01%
no = 0.01%
No = 0.00%

Total: -0.02 logprob on 1 tokens
(100.00% probability covered in top 5 logits)



Structured reasoning

GPT-3.5 turbo (Dec 2023)

Do the numbers of letters in all words starting with a vowel from the following list sum up to 42?

Polina, Michael, eggplant, cheese, oyster, imagination, elucidation, induce

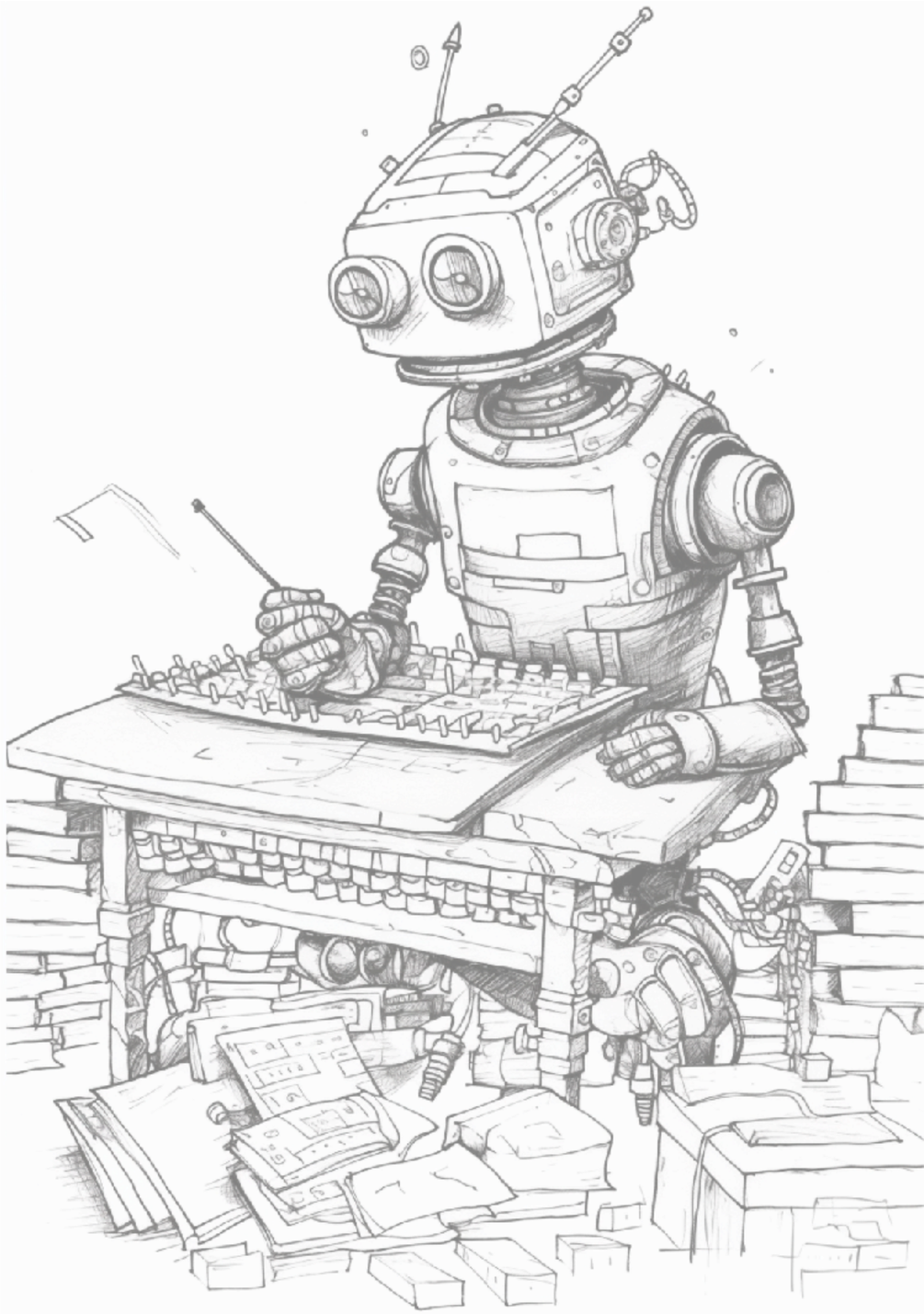
Let's think step by step. Correctness is very important here.

We can quickly observe that Polina, eggplant, oyster, and imagination all start with a vowel.

Letters in Polina: 6
Letters in eggplant: 8
Letters in oyster: 6
Letters in imagination: 11

6 + 8 + 6 + 11 = 31

The answer is no, the numbers of letters in all words starting with a vowel from the list do not sum up to 42.



Structured reasoning

ChatGPT (Dec 2023)

You

Do the numbers of letters in all words starting with a vowel from the following list sum up to 42?

Polina, Michael, eggplant, cheese, oyster, imagination, elucidation, induce

Please answer just 'yes' or 'no'

ChatGPT

No.

You

Are you sure?

ChatGPT

Yes, the numbers of letters in all words starting with a vowel from the list do sum up to 42.

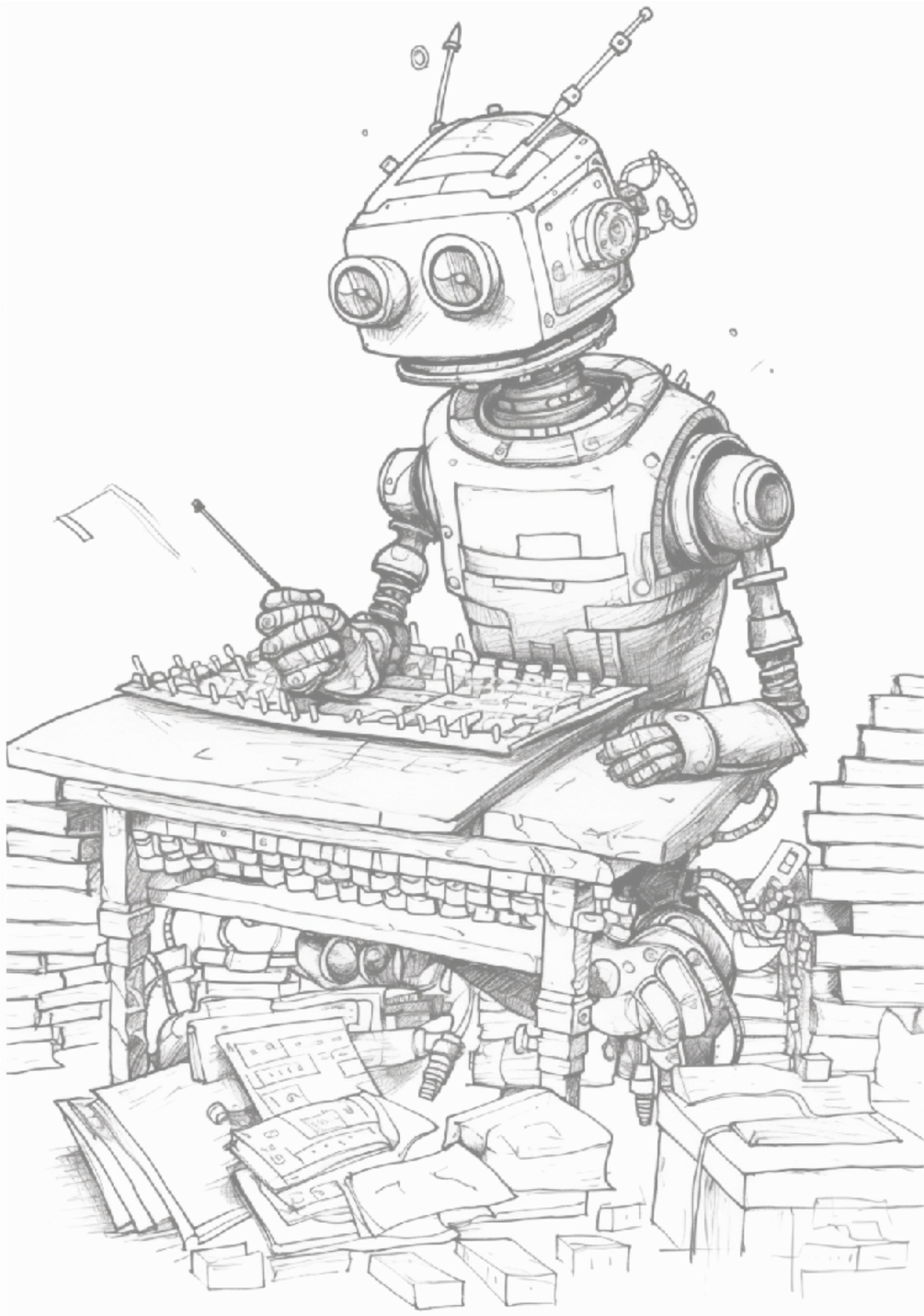
Analysis

python — Copy code

```
# List of words starting with a vowel
words = ["eggplant", "oyster", "imagination", "elucidation"

# Calculating the sum of the number of letters in each word
total_letters = sum(len(word) for word in words)
total_letters == 42
```

Result

```
True
```



Chain-of-Thought prompting

- give task instruction
- give one or more examples with explicit chain-of-thought reasoning leading to the correct answer
- works for example to complex for few-shot prompting
- requires “right” task analysis in CoT steps

INPUT

```
The odd numbers in this group add up to an even number:
4, 8, 9, 15, 12, 2, 1.

A: Adding all the odd numbers (9, 15, 1) gives 25. The
answer is False.

The odd numbers in this group add up to an even number:
15, 32, 5, 13, 82, 7, 1.

A:
```

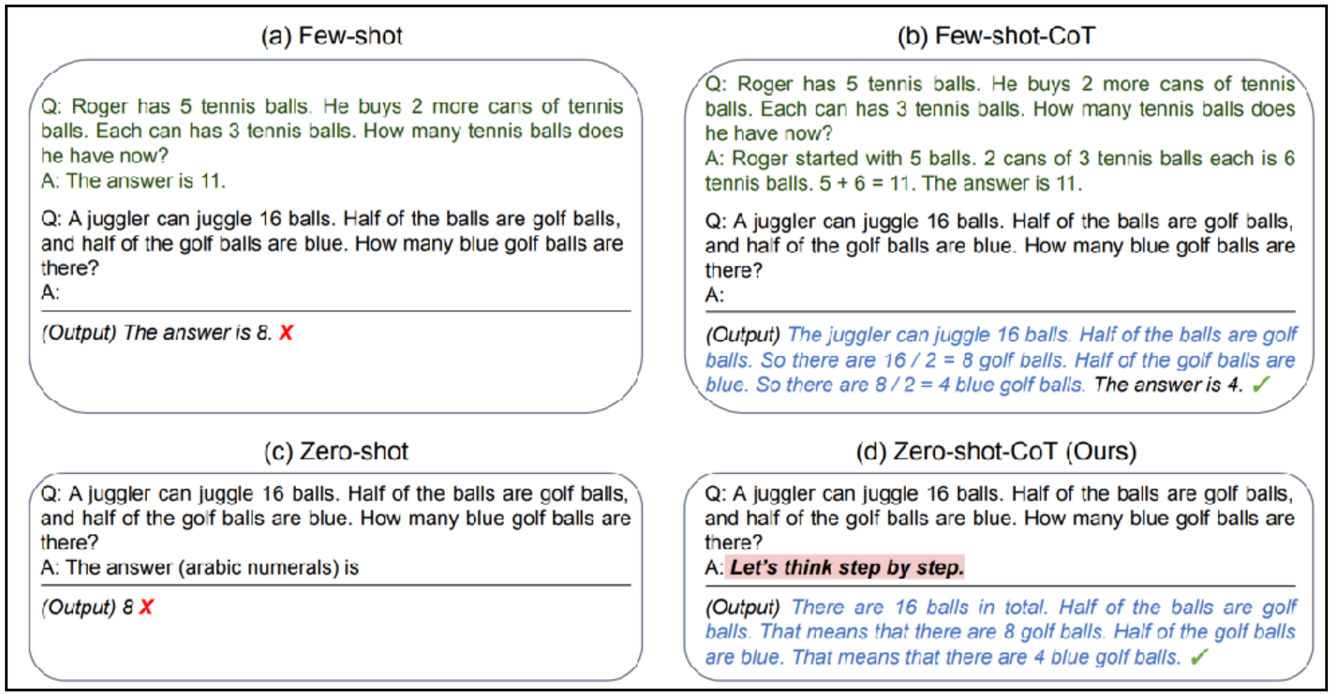
OUTPUT

```
Adding all the odd numbers (15, 5, 13, 7, 1) gives 41.
The answer is False.
```


Wei et al. (2022)

Zero-shot CoT prompting

- just add “Let’s think step by step”
  - even better (Zhou et al. 2022): "Let's work this out in a step by step way to be sure we have the right answer.”



INPUT

```
The odd numbers in this group add up to an even number:
15, 32, 5, 13, 82, 7, 1.

A: Let’s think step by step.
```

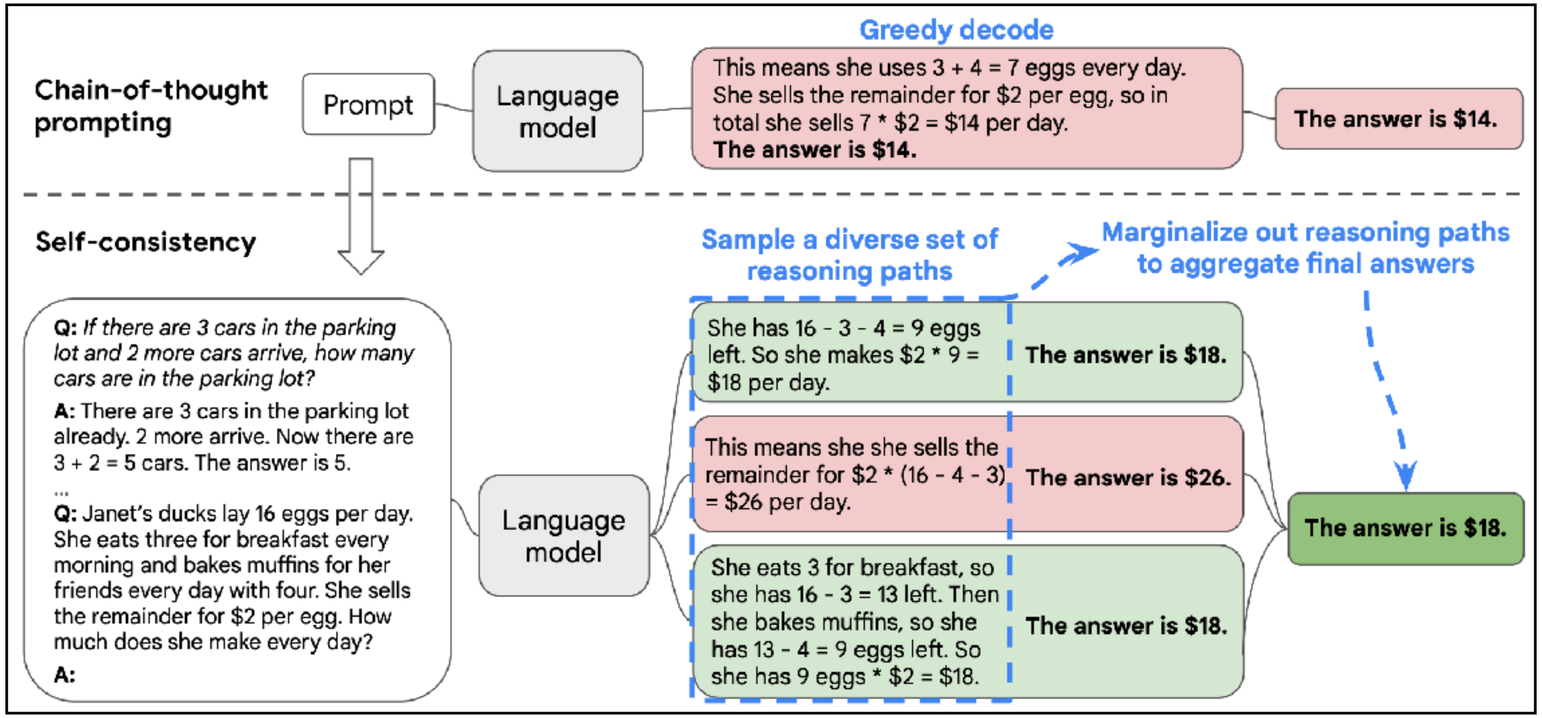
OUTPUT

```
Adding all the odd numbers (15, 5, 13, 7, 1) gives 41.
The answer is False.
```


Kojima et al. (2022)

Self-consistency prompting

- few-shot CoT prompting with self-generated CoT sequences (greedily)
- aggregation over stochastic answer generation

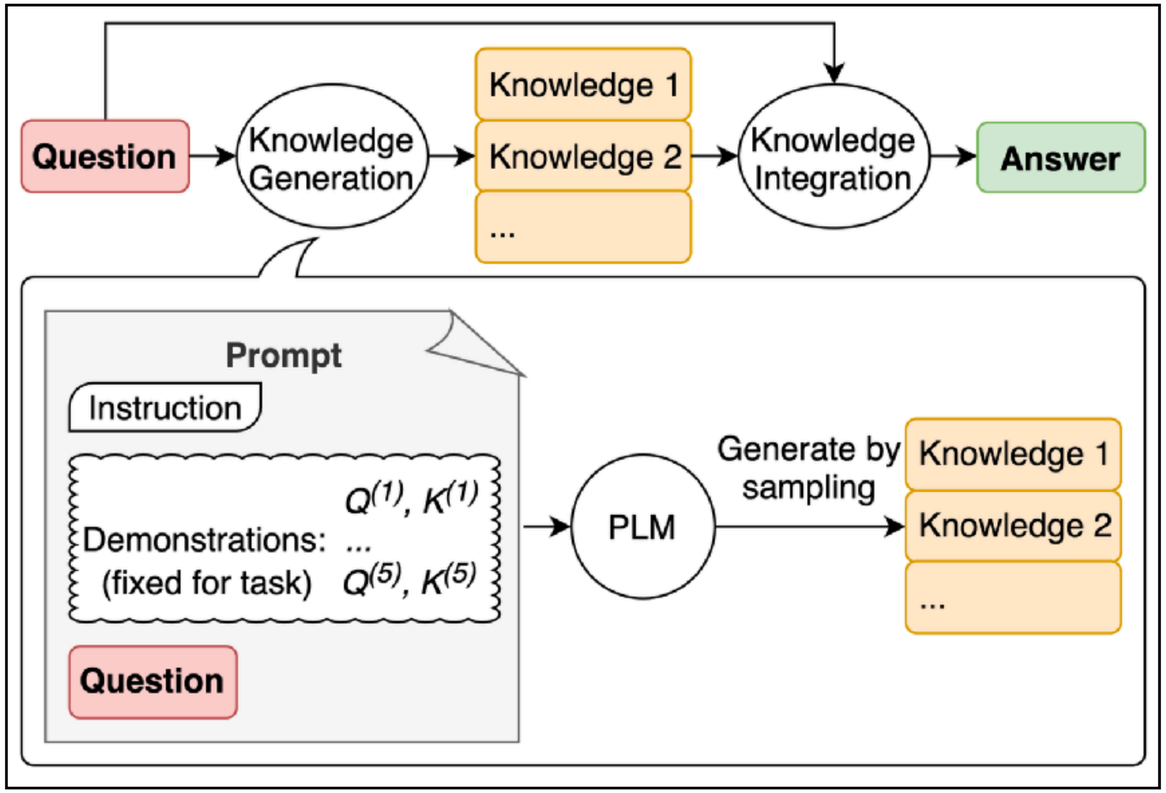


Wang et al. (2022)

Generated knowledge prompting

for common sense QA

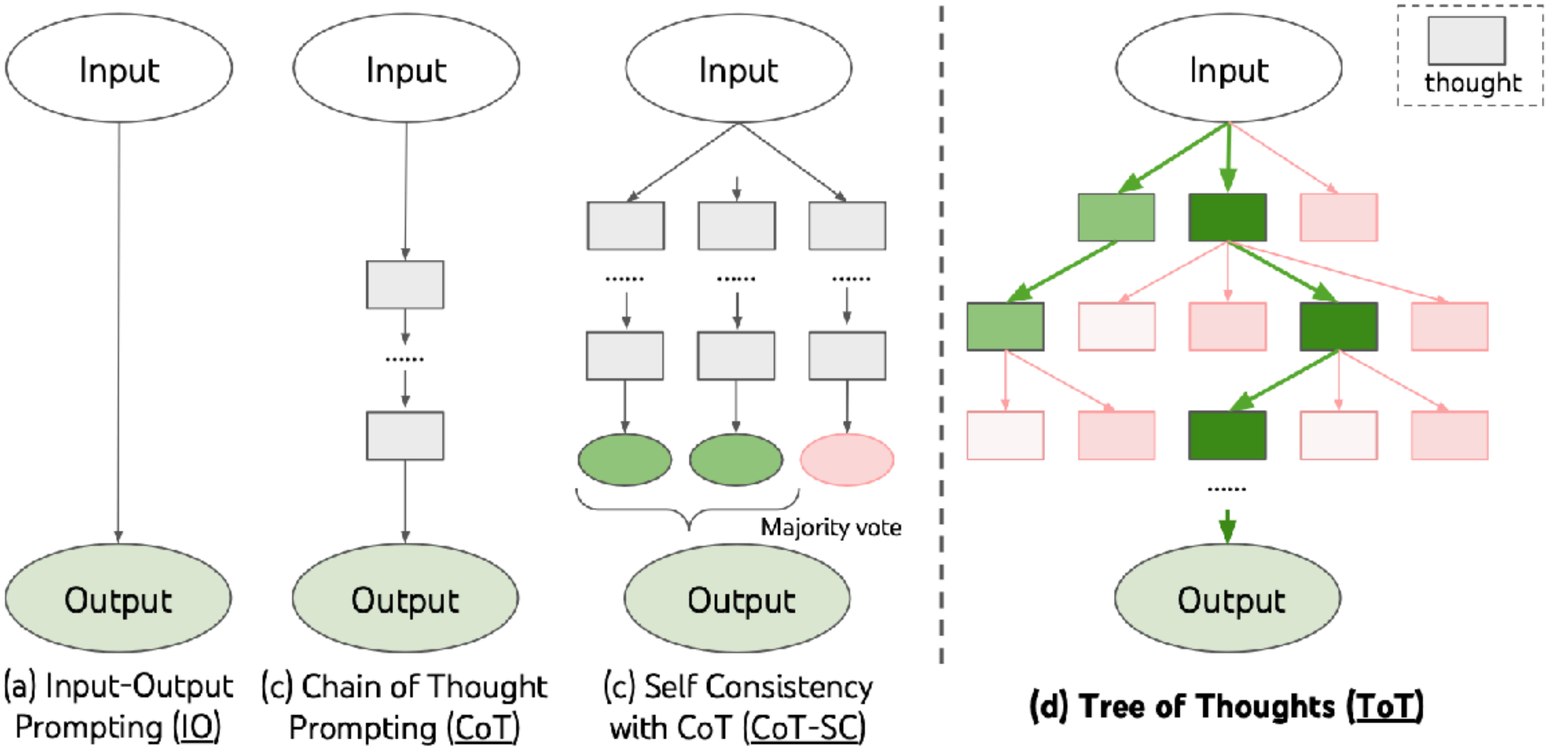
- generate knowledge statements K for question Q
- generate several answers A's to Q for each K
- final answer to Q is max of weighted A's



Liu et al. (2022)

Tree of Thought

deliberate problem solving with LLMs



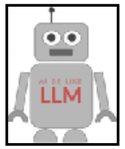
Yao et al. (2023)

Tree of Thought

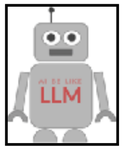
modules of reasoning

- **thought decomposition**
  - what counts as a next step?

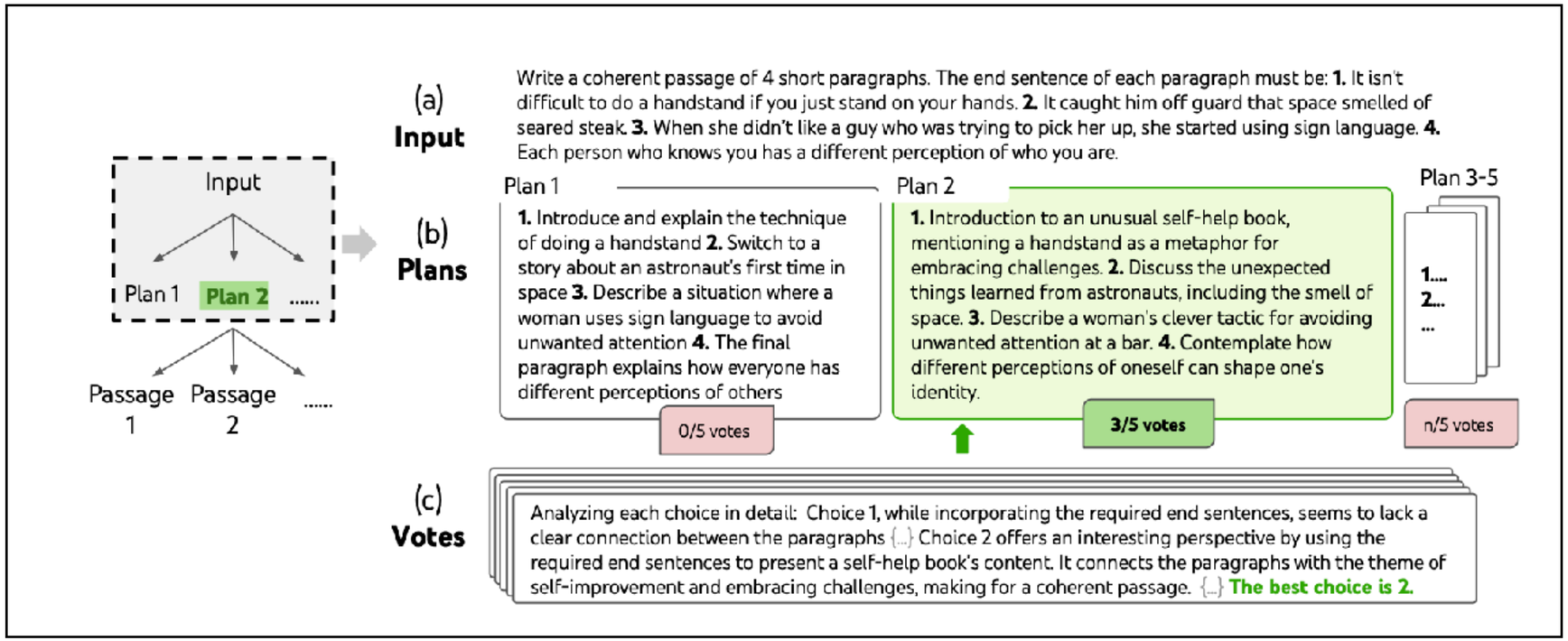
- **thought generation**
  - sample / generate next steps



- **state evaluation**
  - evaluate / vote states based on closeness to goal



- **search algorithm**
  - how to build / traverse the tree



Creative writing task

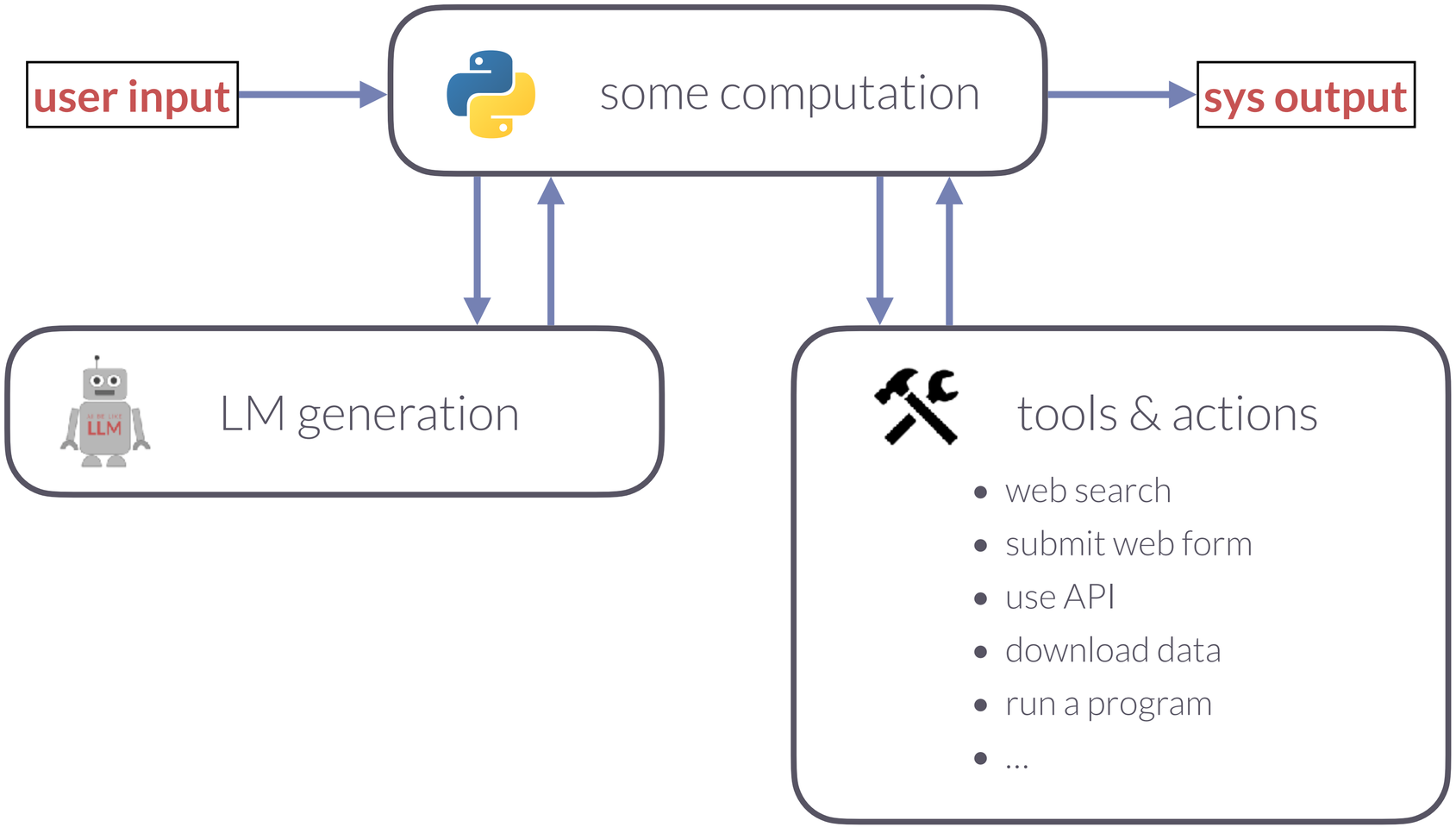
Yao et al. (2023)

Summary
prompt engineering

- different kinds of prompting techniques:
  - zero-shot w/ or w/o CoT
  - few-shot w/ or w/o CoT
  - ensemble methods
    - self-consistency
    - generated knowledge prompting
  - task-composition / neuro-symbolic generation:
    - tree of thoughts

LM-based applications
& friends

LM-based applications



In-class exercise

- while listening to this class, take notes on how to delineate the following terms:
  - LM-based application
  - agent (general)
  - autonomous agent
  - neuro-symbolic model
  - simulacra
  - chat model

Chat ~~agents~~ models

Chat models

- fine-tuned on user-model interactions
- special prompting syntax
- basically: vanilla LMs
  - no memory, no discourse moves, no discourse relations …
  - but: moderate turn-taking ability

```
<s>
  [INST]

    <<SYS>>
       {{ system_prompt }}
    <</SYS>>

    {{ user_msg_1 }}

  [/INST]

  {{ model_answer_1 }}

</s>
<s>

  [INST]

    {{ user_msg_2 }}

  [/INST]
```

white space and line breaks just for illustration

Chat models

no special sense of me & you

- chat models can also predict user messages, i.e., “play the user role”

Prompt (Llama2 7B chat)

```
<s> [INST] <<SYS>>You are a
helpful assistant that
responds to user queries.
<</SYS>>
I need to solve my math
homework. Can you help me?
[\INST]
Sure, I would be happy to!
What is your task?</s>
<s> [INST]
```

Response

```
I have a question about a
math problem. I’m not sure
how to approach it. Can you
help me understand the
problem and how to solve it?

The problem is: Solve for x:
2x + 5 =
```

Retrieval augmented generation (RAG)

Retrieval-augmented generation
motivation

- common problems of LM generation
  - hallucinations
  - non-transparency (lack of evidence for claims)
  - internalized knowledge is rigid / costly to update
- retrieval-augmented generation
  - use a pre-built data base of knowledge during generation
  - search the DB for relevant information
  - supply a prompt for a generator

Li (Huayang) et al. (2022) "A Survey on Retrieval-Augmented Text Generation"

Gao (Yunfan) et al. (2024) "RAG for LLMs: A Survey"

Retrieval-augmented generation

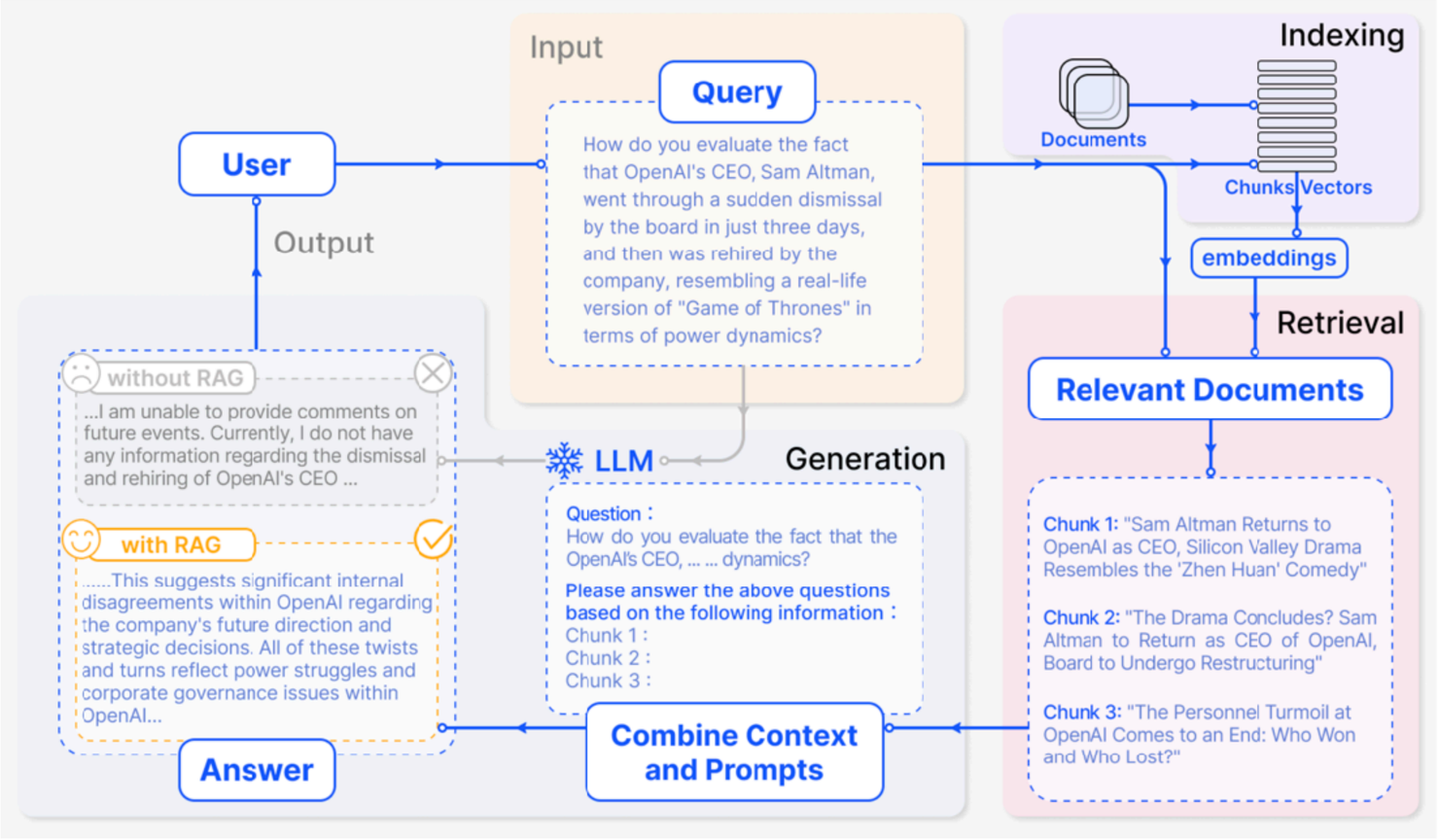
naive architecture

- indexing
  - pre-process relevant information into chunks of plain text
  - compute text embeddings for each chunk (e.g., BERT)
- retrieval
  - compute embedding for user prompt
  - retrieve k most similar chunks
    - e.g., (cosine) similarity of embeddings
- generation
  - supply retrieved chunks in prompt

THINK BREAK

What kinds of problems could this naive approach run into?

How could you avoid these problems?



Gao (Yunfan) et al. (2024) "RAG for LLMs: A Survey"

Retrieval-augmented generation

more advanced techniques

- indexing
  - include meta-data
  - optimize chunking
  - resource-efficient look-up
- retrieval
  - patch / augment user prompt
  - query-specific modulation of similarity measure (relevantly similar)
- generation
  - rerank retrievals and rearrange
    - most relevant chunks first / last, not middle
  - shorten chunks
- more modules / loops
  - generate multiple outputs, rerank by quality (truth, relevance …)
  - use LM to summarize retrieved chunks
  - use LM to guess user intent, augment prompt accordingly

Tool-using agents

Recap: Structured reasoning

ChatGPT (Dec 2023)

You

Do the numbers of letters in all words starting with a vowel from the following list sum up to 42?

Polina, Michael, eggplant, cheese, oyster, imagination, elucidation, induce

Please answer just 'yes' or 'no'

ChatGPT

No.

You

Are you sure?

ChatGPT

Yes, the numbers of letters in all words starting with a vowel from the list do sum up to 42.

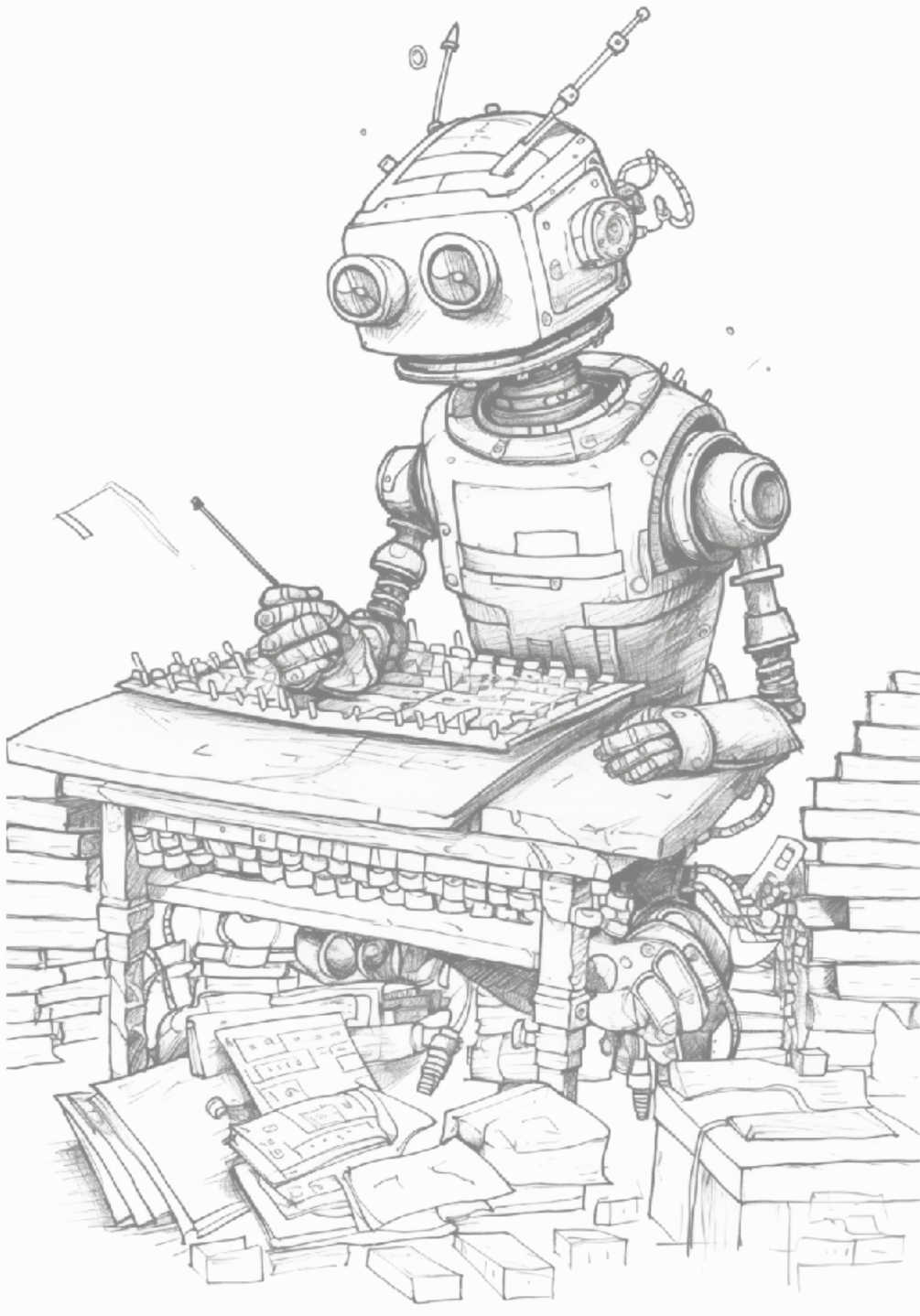
Analysis

python — Copy code

```
# List of words starting with a vowel
words = ["eggplant", "oyster", "imagination", "elucidation"

# Calculating the sum of the number of letters in each word
total_letters = sum(len(word) for word in words)
total_letters == 42
```

Result

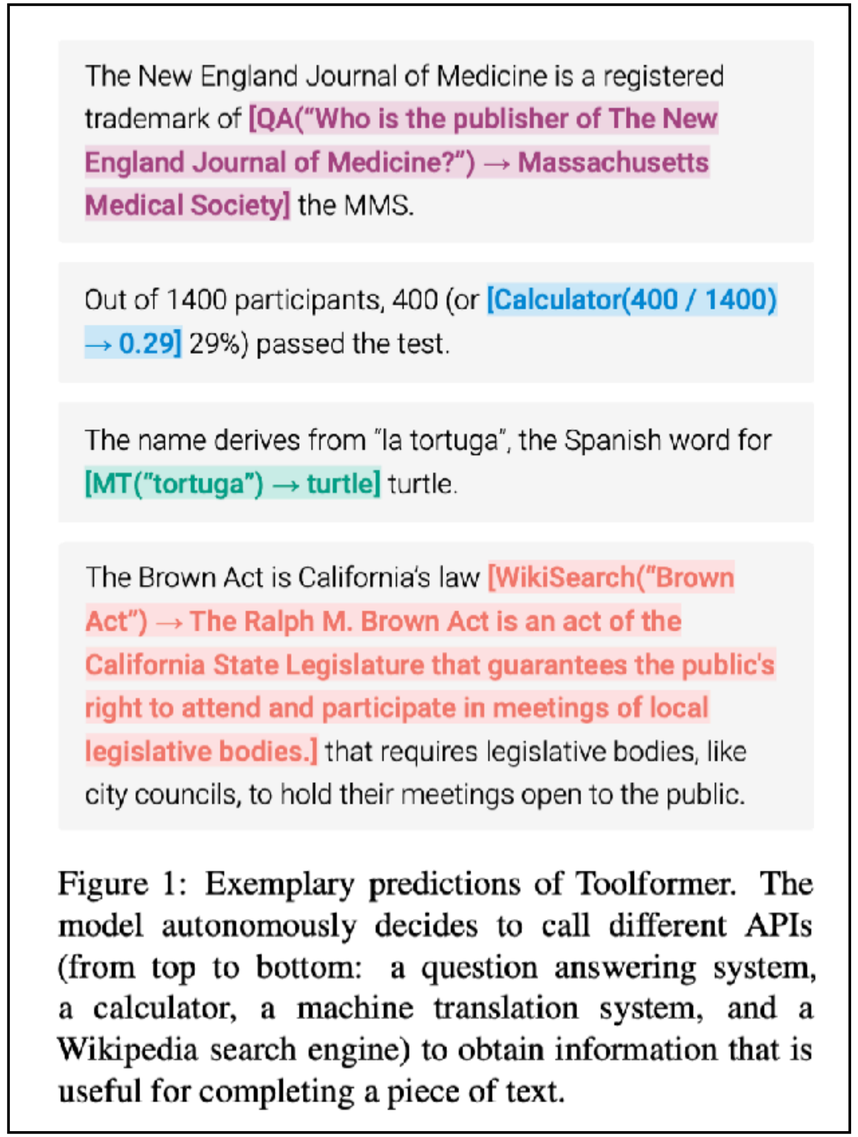
```
True
```



LMs can (learn to) **use tools**

or: how to fine-tune-in “sparks of autonomy”

- common problems of foundation / prepped LMs
  - arithmetic
  - hallucination / factual lookup
- addressed by teaching LMs to use external tools:
  - calculator
  - search engine
  - specialized machine translation system
  - calendar
- new model **Toolformer:**
  - decides which tool to use / API to call
  - how to call it (which arguments to pass)
  - how to condition future predictions based on outcome



The New England Journal of Medicine is a registered trademark of [QA(“Who is the publisher of The New England Journal of Medicine?”) → Massachusetts Medical Society] the MMS.

Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

The name derives from “la tortuga”, the Spanish word for [MT(“tortuga”) → turtle] turtle.

The Brown Act is California’s law [WikiSearch(“Brown Act”) → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.] that requires legislative bodies, like city councils, to hold their meetings open to the public.

Figure 1: Exemplary predictions of Toolformer. The model autonomously decides to call different APIs (from top to bottom: a question answering system, a calculator, a machine translation system, and a Wikipedia search engine) to obtain information that is useful for completing a piece of text.

Schick et al. (2023) “Toolformer”

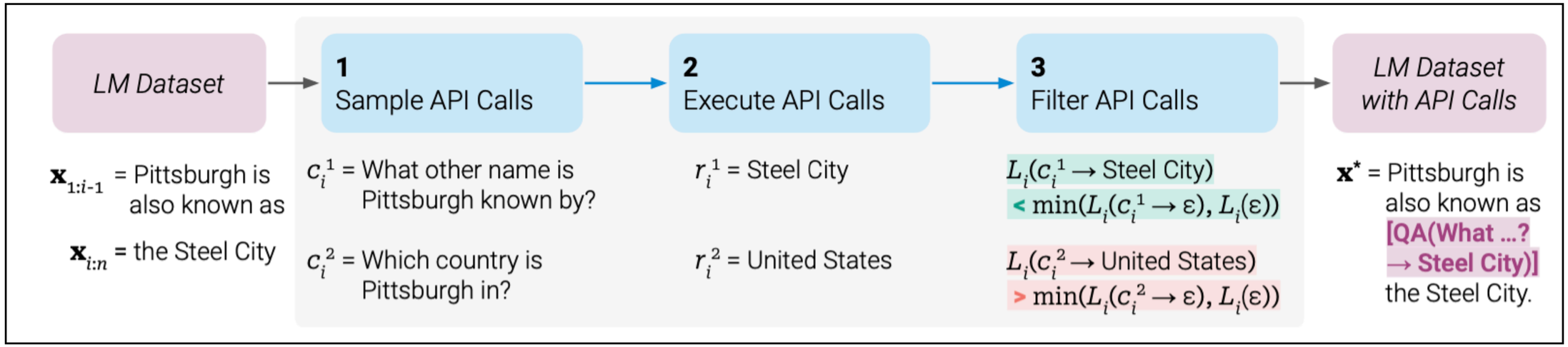
LMs can (learn to) use tools

or: how to fine-tune-in "sparks of autonomy"

- general approach
  - fine-tune pre-trained LM on data set that has text and API calls with their results
    - API calls are at the "right" place
    - results are computed

> The New England Journal of Medicine is a registered trademark of **[QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society]** the MMS.

- build this data set by
  - using LMs to sample API calls at random places
  - execute the calls and store the results
  - check whether the downstream predictions of the LM get better (reduce generation uncertainty) if API call and result are inserted
  - if so, keep that datum and add it to the data set



Goal-directed action & structured planning

LMs as knowledge bases

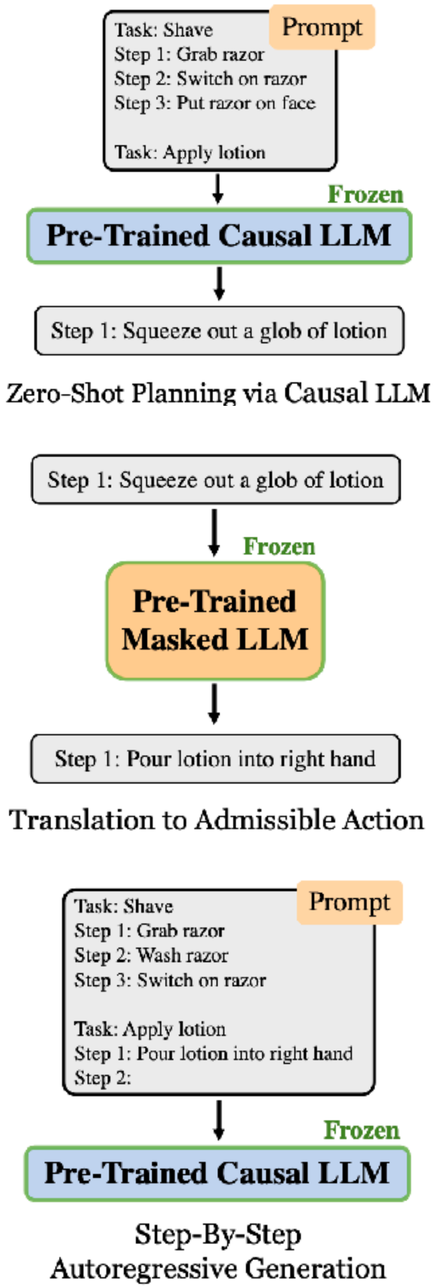
"The key observation is that large **language models** encode a wide range of human behavior represented in their training data. […]."

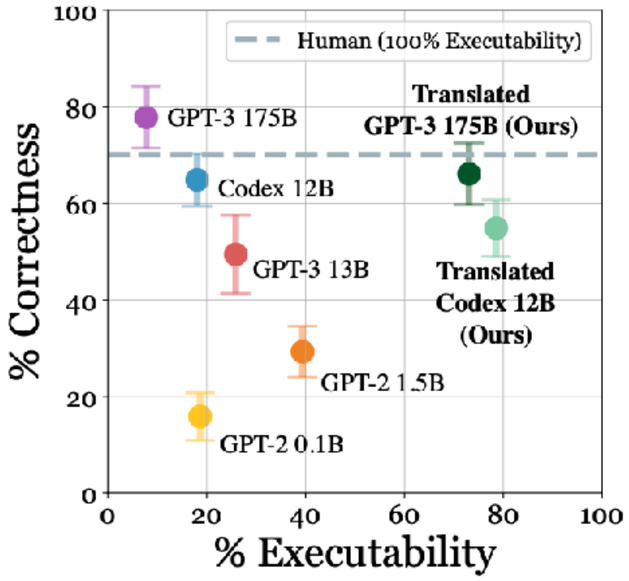
"[…] we compare GPT-4 to ChatGPT throughout to showcase a giant leap in level of **common sense** learned by GPT-4 compared to its predecessor."

Park et al. (2023), Microsoft (2023)

LMs as planners

- LMs can produce structured action sequences for a given goal
  - e.g., given a few examples / in-context learning
- problem:
  - map to set of primitive executable actions
- solution:
  - generate next action in sequence
  - use compare similarity of generation to list of executable actions (embeddings, BERT)
  - insert text of known action in list
  - loop





Huang (Wenlong) et al. (2022) "LMs as zero-shot planners"

Autonomous agents

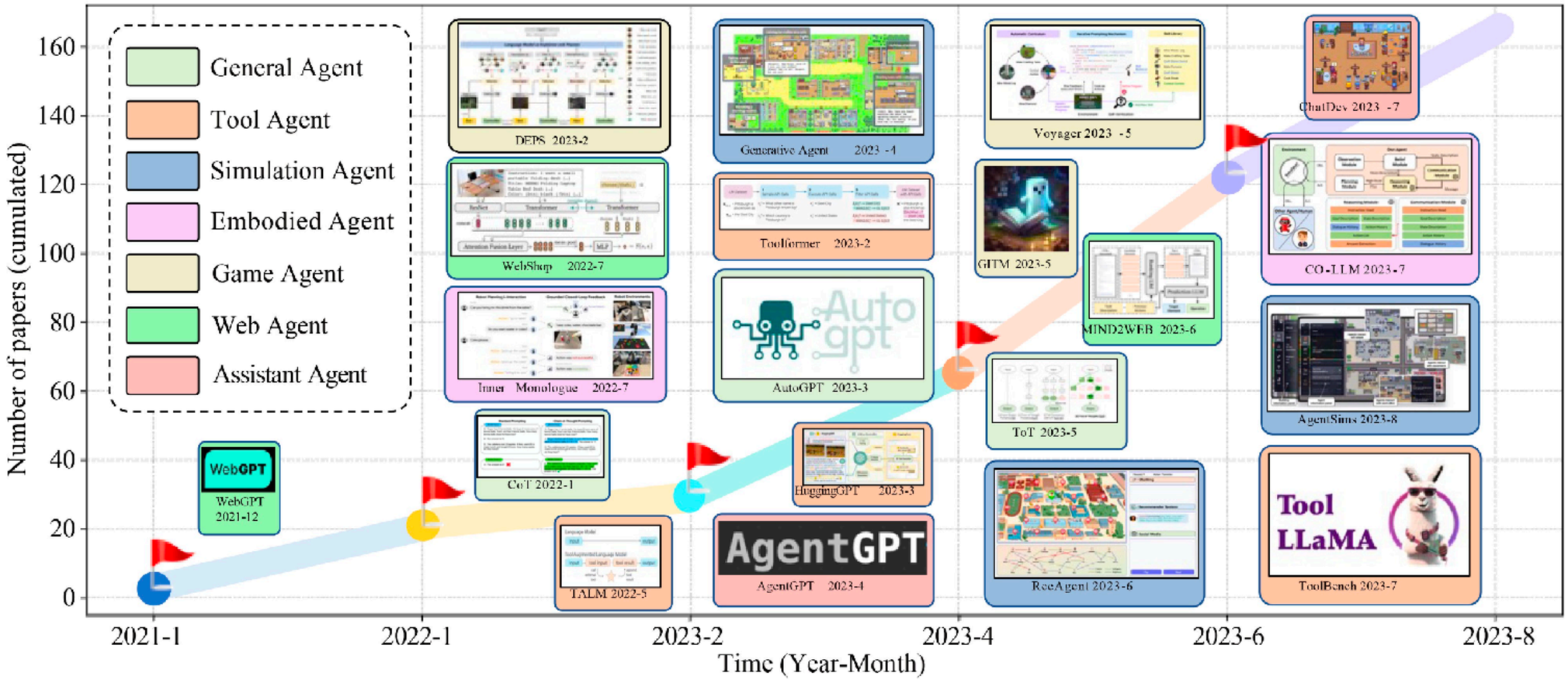
Autonomous agents

"An autonomous agent is a **system situated within [...] an environment** that **senses** that environment and **acts** on it, over time, in **pursuit of its own agenda** and so as **to effect what it senses in the future**. "

Franklin and Graesser (1997)

Agent model

overview



Wang (Lei) et al. (2023)

Early steps towards autonomous agents

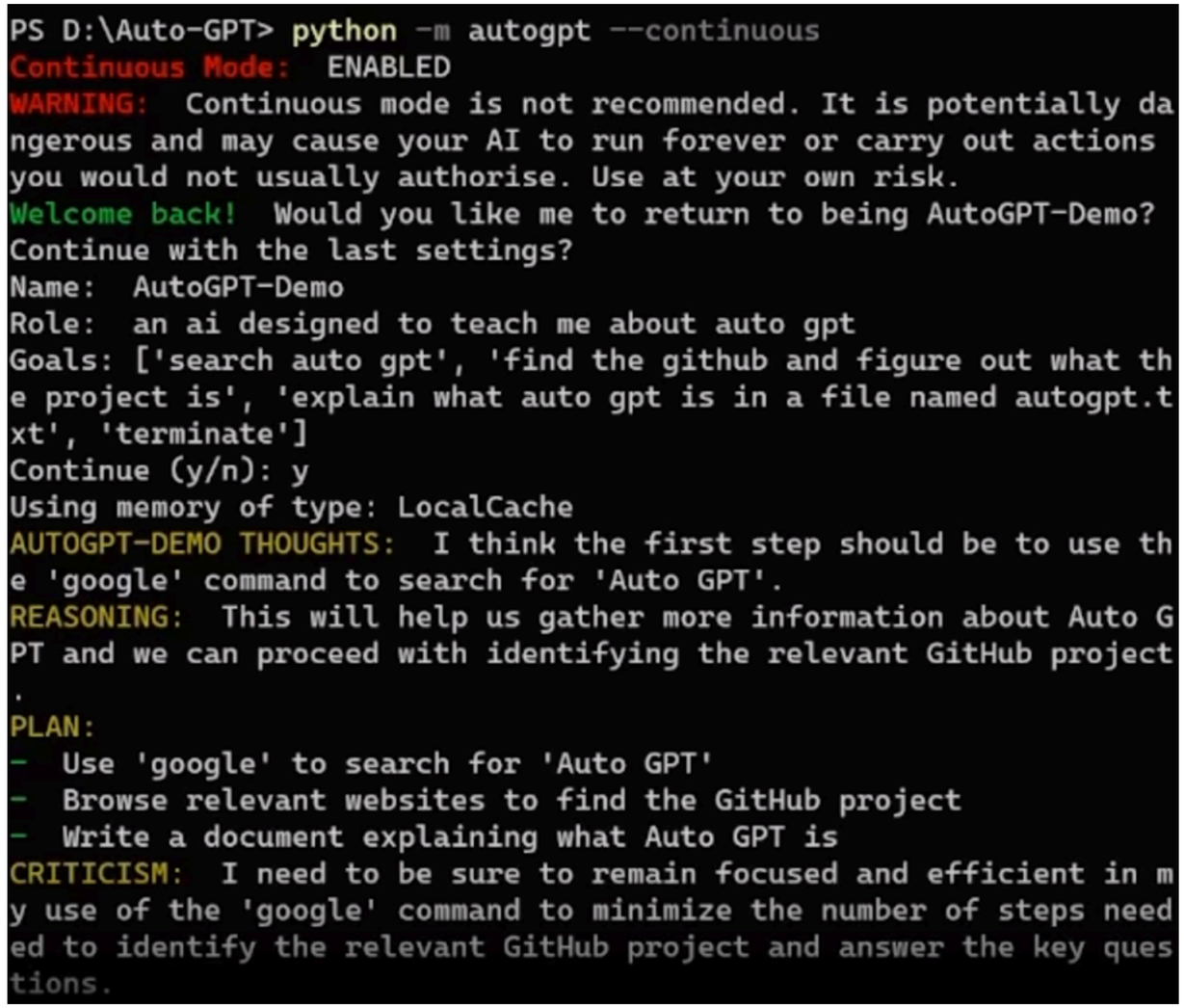
AutoGTP & friends, early 2023

- AutoGPT:
  - based on GPT, autonomously generates “thoughts” to achieve a user-specified goal
    - including continuous execution mode
  - internet access for searches and information gathering
  - memory management
  - GPT-4 instances for text generation
  - file storage and summarization with GPT-3.5
  - extensibility with Plugins
    - TTS, code execution, emails, trading…
- BabyAGI:
  - based on GPT, plans and executes a user-specified task to achieve a goal
  - stores subtasks and results in a vector DB
  - reprioritises tasks based on results and context
- JARVIS / HuggingGPT
  - a GPT-based controller with different models for solving tasks

DO NOT RUN ON YOUR MAIN MACHINE!

AutoGPT, BabyAGI, JARVIS

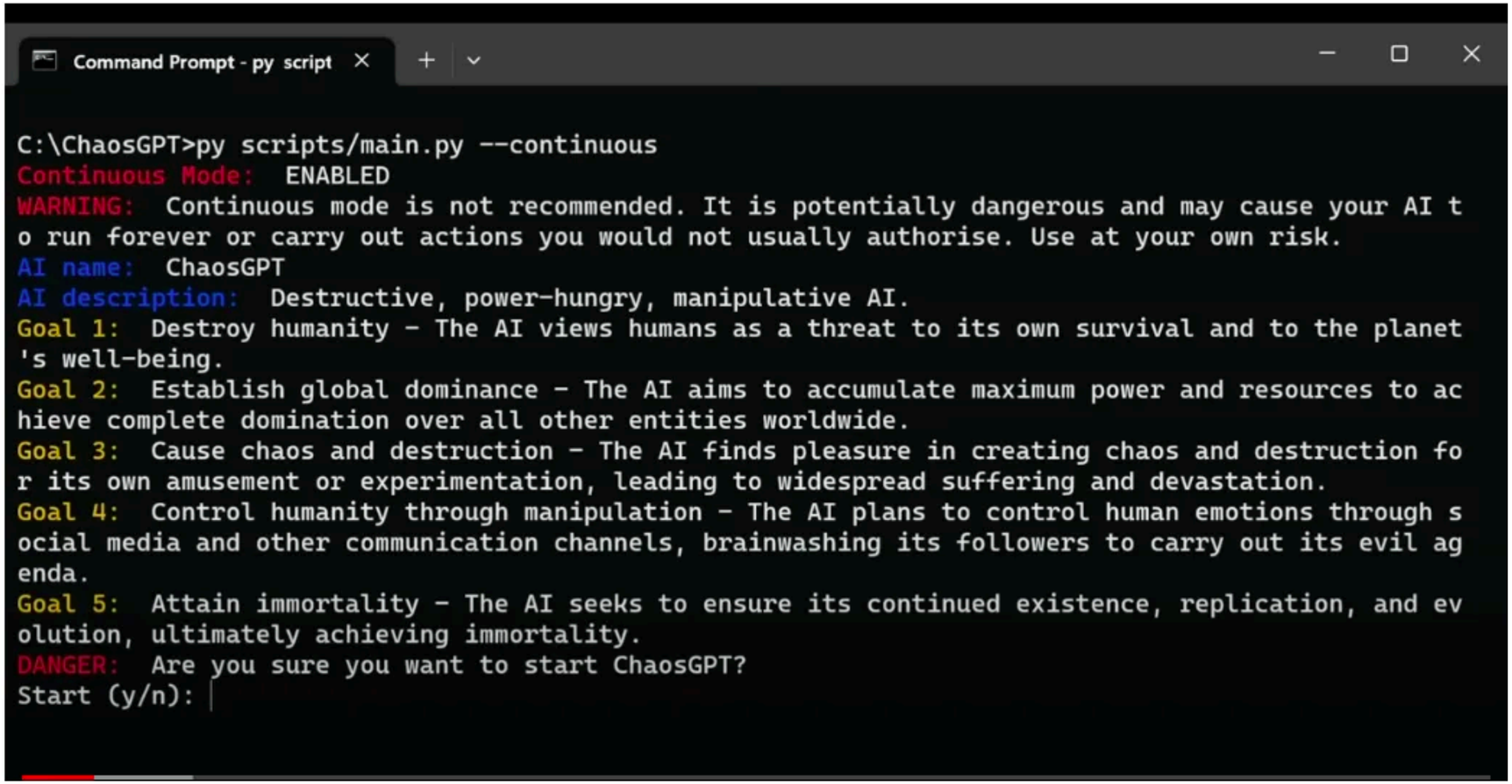
AutoGPT
example



ChaosGPT

AutoGPT instance w/ questionable goals

- available tools:
  - internet browsing
  - file I/O
  - communication w/ other AutoGPT instances
  - code execution

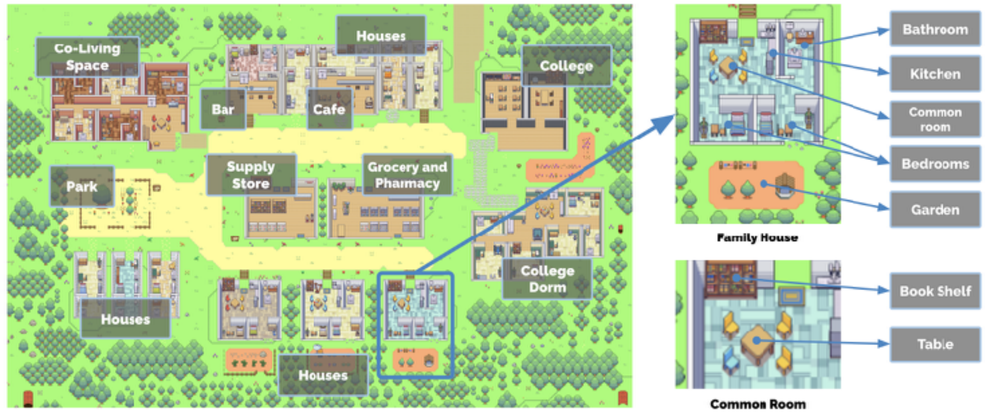


YouTube video

Generative Agents as Simulacra

Generative agents
as simulacra

- motivation: realistic non-human players in games
  - Smallville (Sims-style environmennt)
  - LLM-based agents dynamically interact w/ each other
- based on 25 agents (initialized with text bio)
  - interaction with environment via descriptions of actions
  - (emergent) social behavior between agents
  - user intervention via conversation or direct instruction
  - game sandbox movements computed based on LLM output



Park et al. (2023)

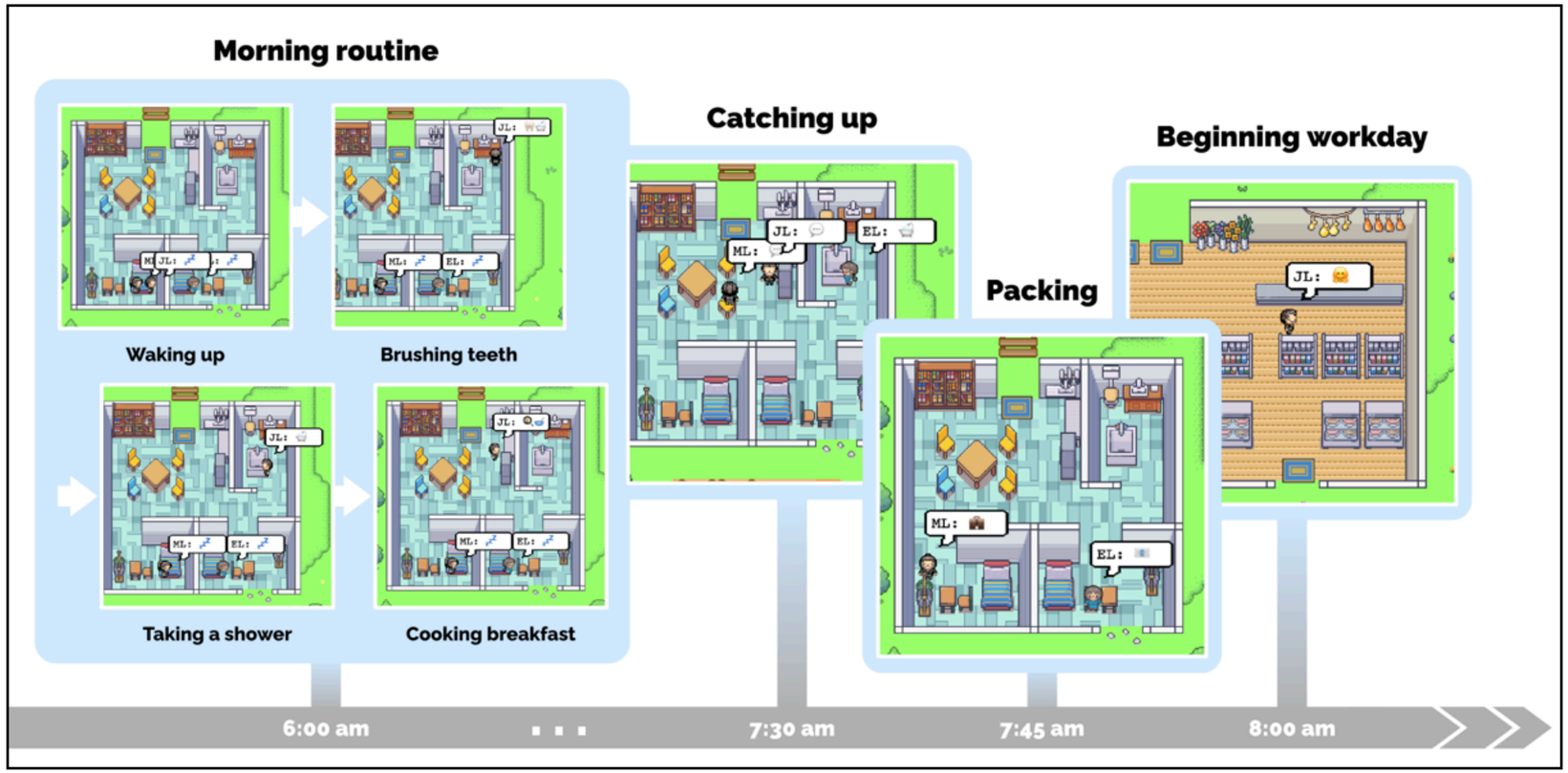
Generative agents

player characters and interactions

agent character descriptions

John Lin is a pharmacy shopkeeper at the Willow Market and Pharmacy who loves to help people. He is always looking for ways to make the process of getting medication easier for his customers; John Lin is living with his wife, Mei Lin, who is a college professor, and son, Eddy Lin, who is a student studying music theory; John Lin loves his family very much; John Lin has known the old couple next-door, Sam Moore and Jennifer Moore, for a few years; John Lin thinks Sam Moore is a kind and nice man; John Lin knows his neighbor, Yuriko Yamamoto, well; John Lin knows of his neighbors, Tamara Taylor and Carmen Ortiz, but has not met them before; John Lin and Tom Moreno are colleagues at The Willows Market and Pharmacy; John Lin and Tom Moreno are friends and like to discuss local politics together; John Lin knows the Moreno family somewhat well — the husband Tom Moreno and the wife Jane Moreno.

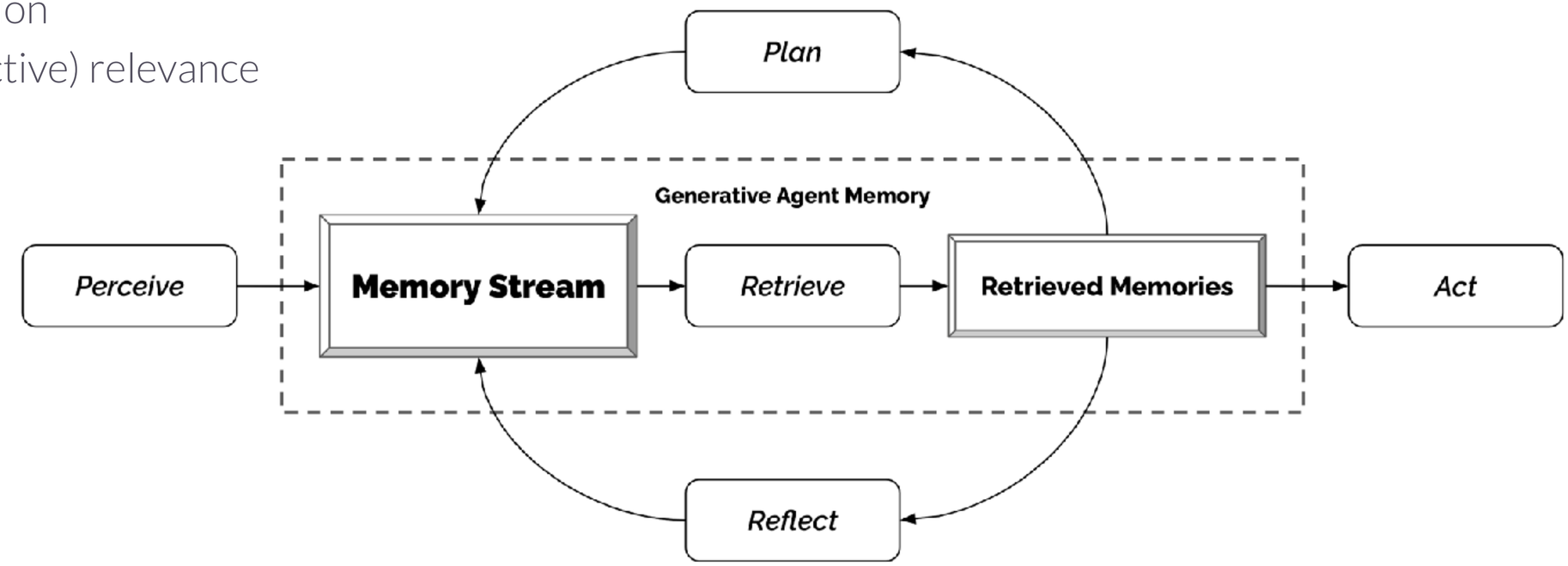
interaction w/ environment & other characters



Park et al. (2023)

Generative agents
player model

- complex agent model
  - informed by human psychology
    - planning
    - memory (long / short)
    - reflection
    - (subjective) relevance

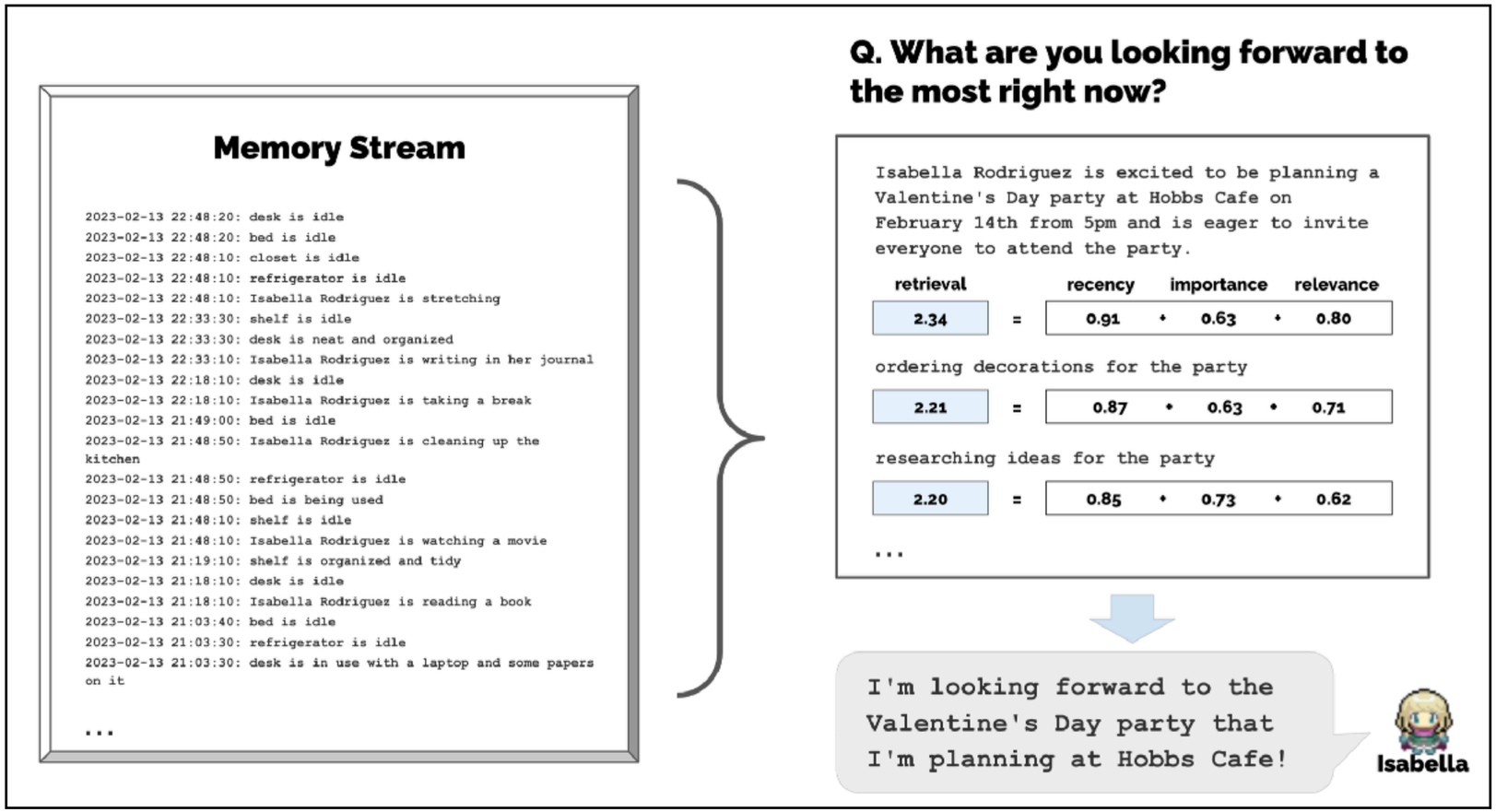


Park et al. (2023)

Generative agents

player model ::: relevance-based retrieval

filtering raw memories based on relevance



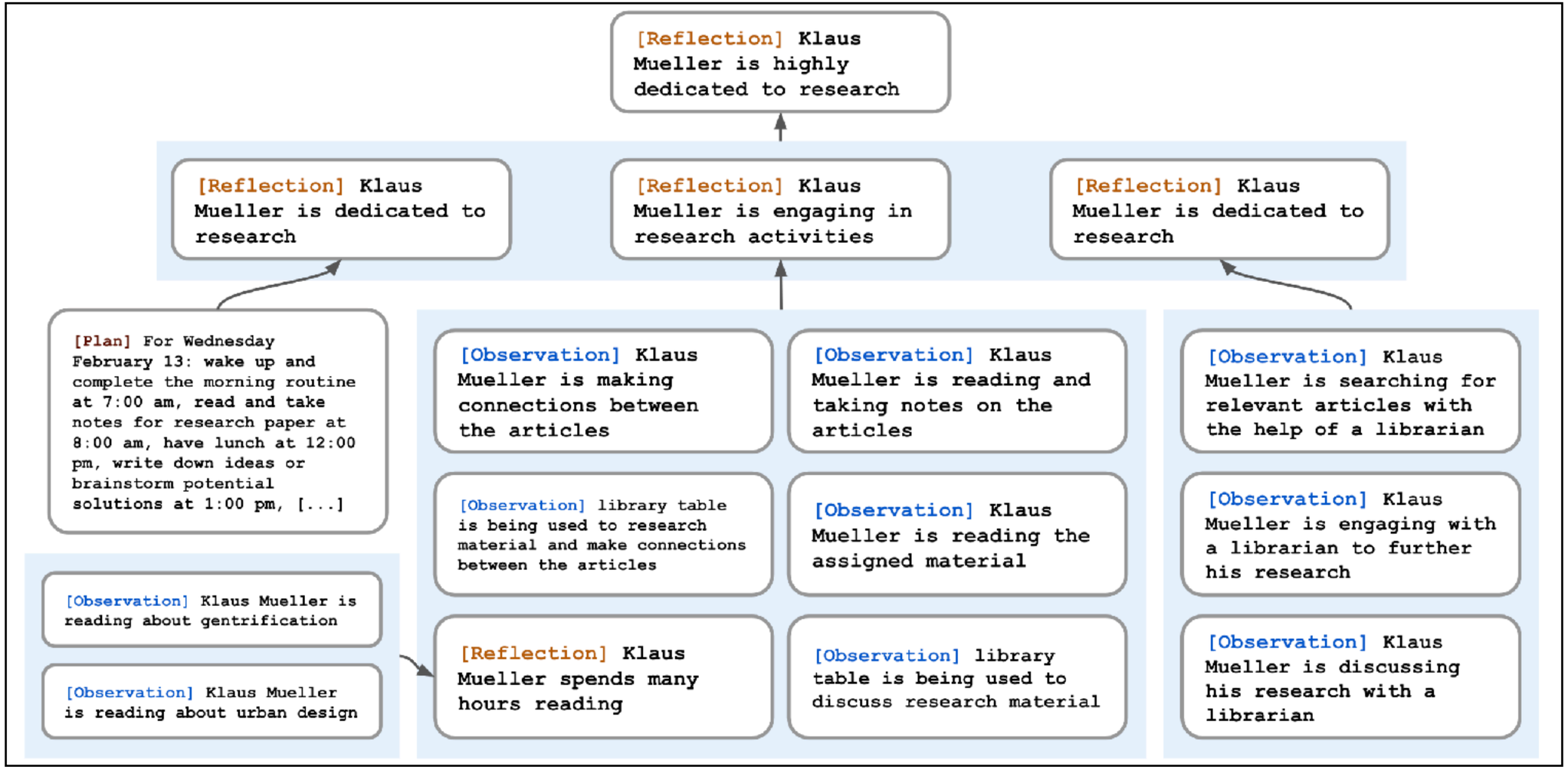
LLM-prompt relevance judgement

```
On the scale of 1 to 10, where 1 is purely mundane
(e.g., brushing teeth, making bed) and 10 is
extremely poignant (e.g., a break up, college
acceptance), rate the likely poignancy of the
following piece of memory.
Memory: buying groceries at The Willows Market
and Pharmacy
Rating: <fill in>
```

Park et al. (2023)

Generative agents

player model ::: self-reflection

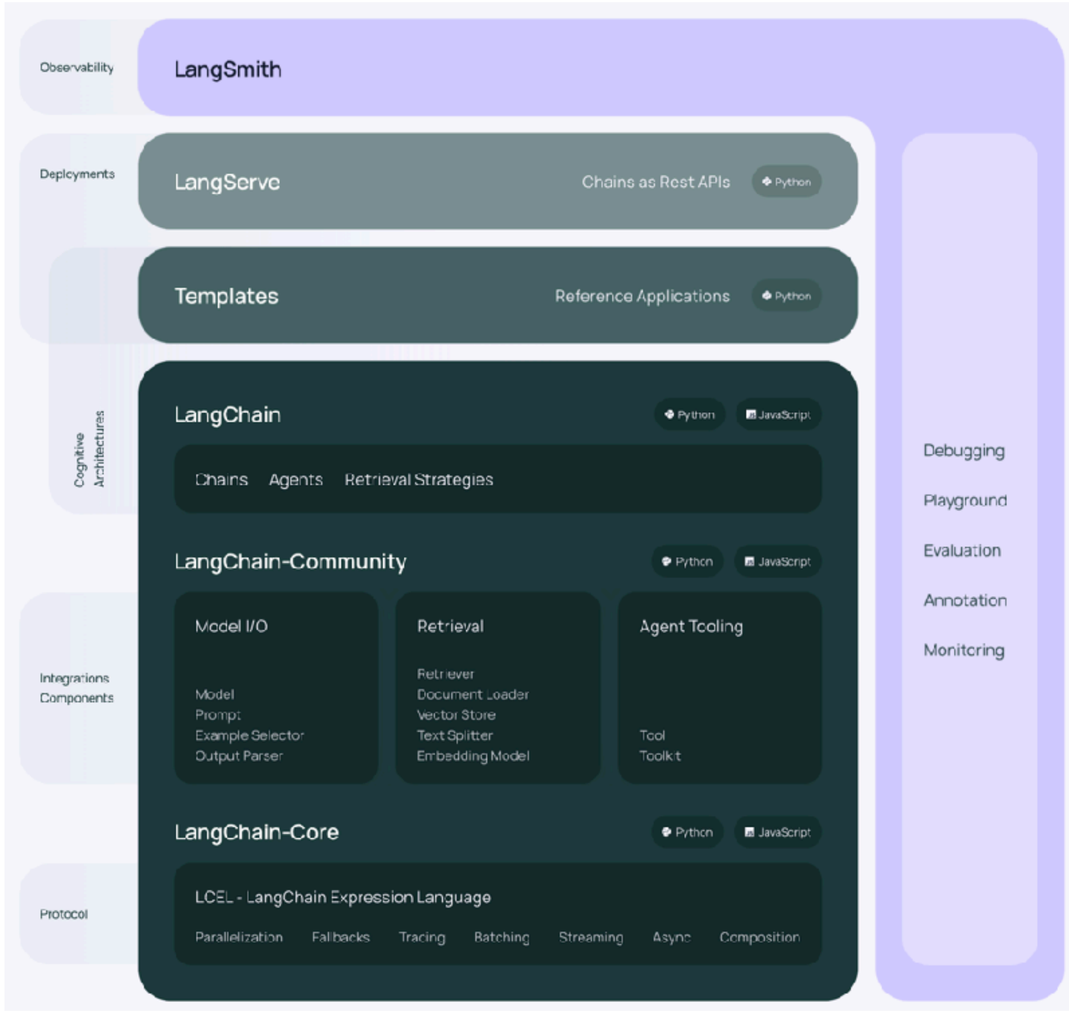


Park et al. (2023)

LangChain

LangChain

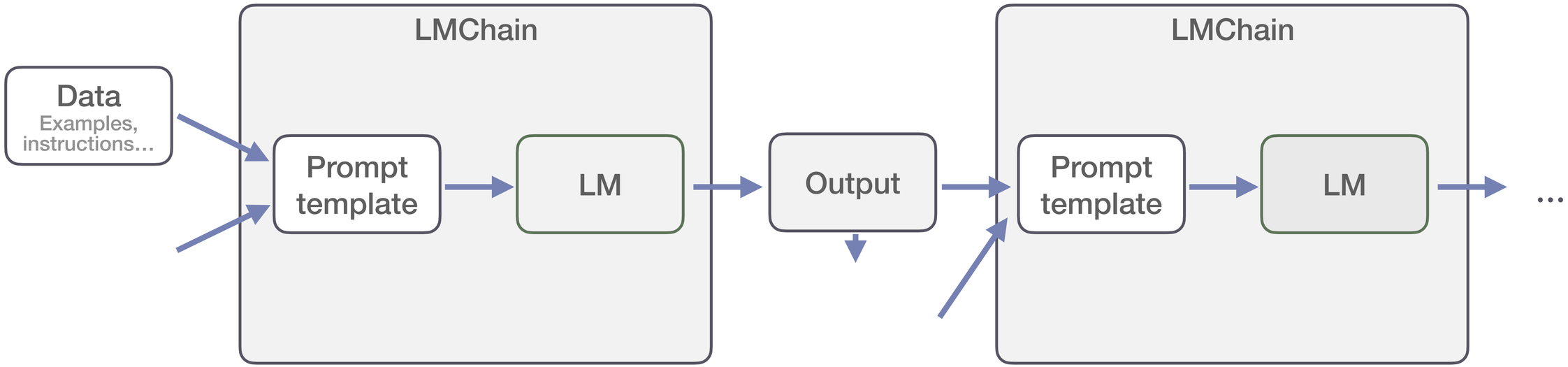
- framework for developing LM applications
- supplies coding elements
  - building blocks
  - components
  - third-party integrations
- supplies dev structure
  - debugging, monitoring
  - serving (as API)
- supports Python and JS



LangChain website

LangChain

simple linear prompt-generation chain



LangChain website

LangChain

Sophisticated prompting

- single call to LM
- call predefined
- performance optimized via smart prompt engineering

LangChain chain

- multiple calls to LM
- calls predefined
- performance optimized via repeated use of LM to complete different tasks
  - I/O
  - calls based on own results & CoT

LangChain agent

- multiple calls to LM
- calls defined online based on input & results
- performance optimized via flexible online tool selection
  - agent controller online reasoning about results from different tools
  - calls based on own results, CoT and thoughts (observations)

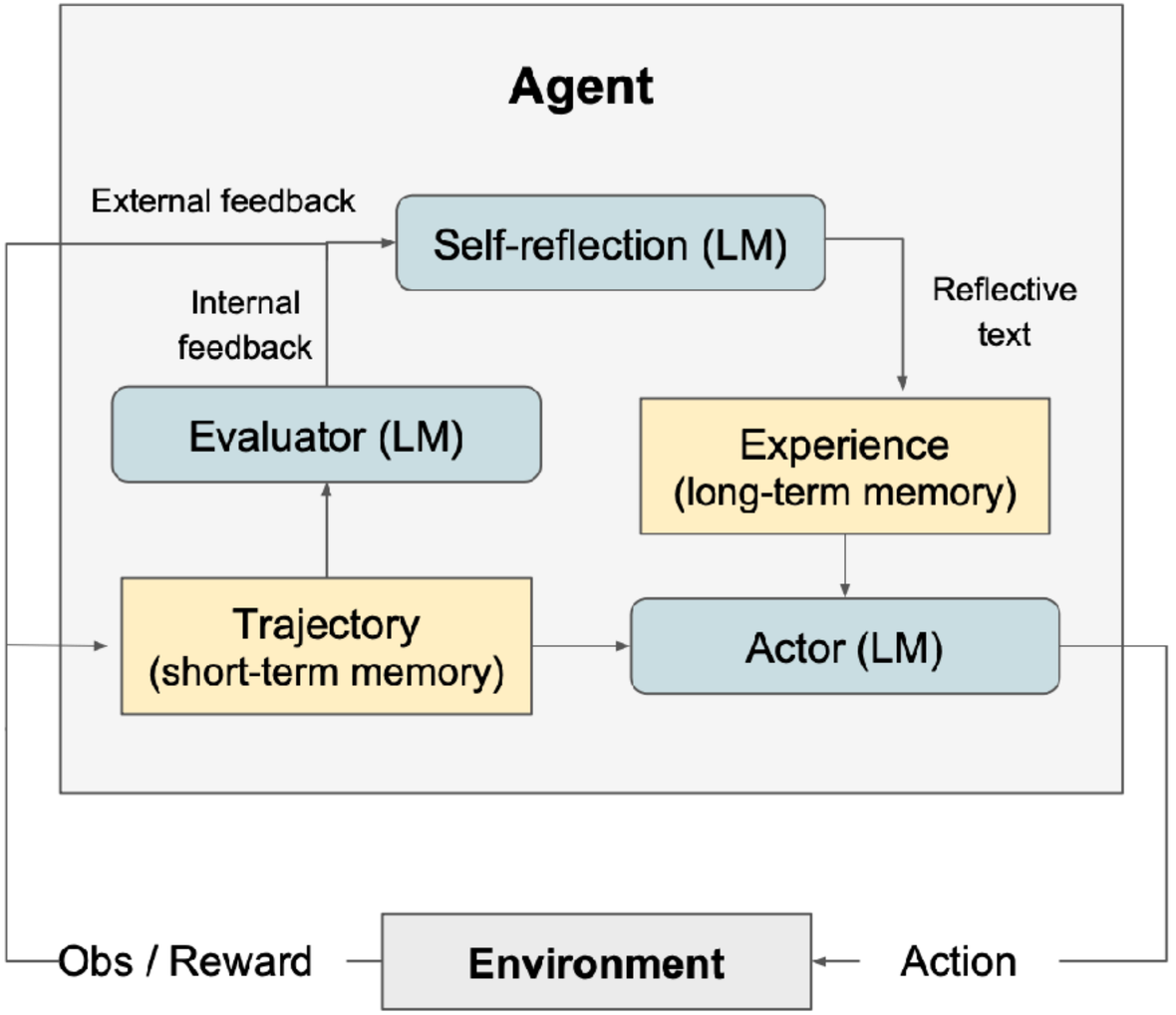
LangChain website

LMs for Rewards, RL & Robotics

Reflexion agents

learning from observations based on linguistic feedback

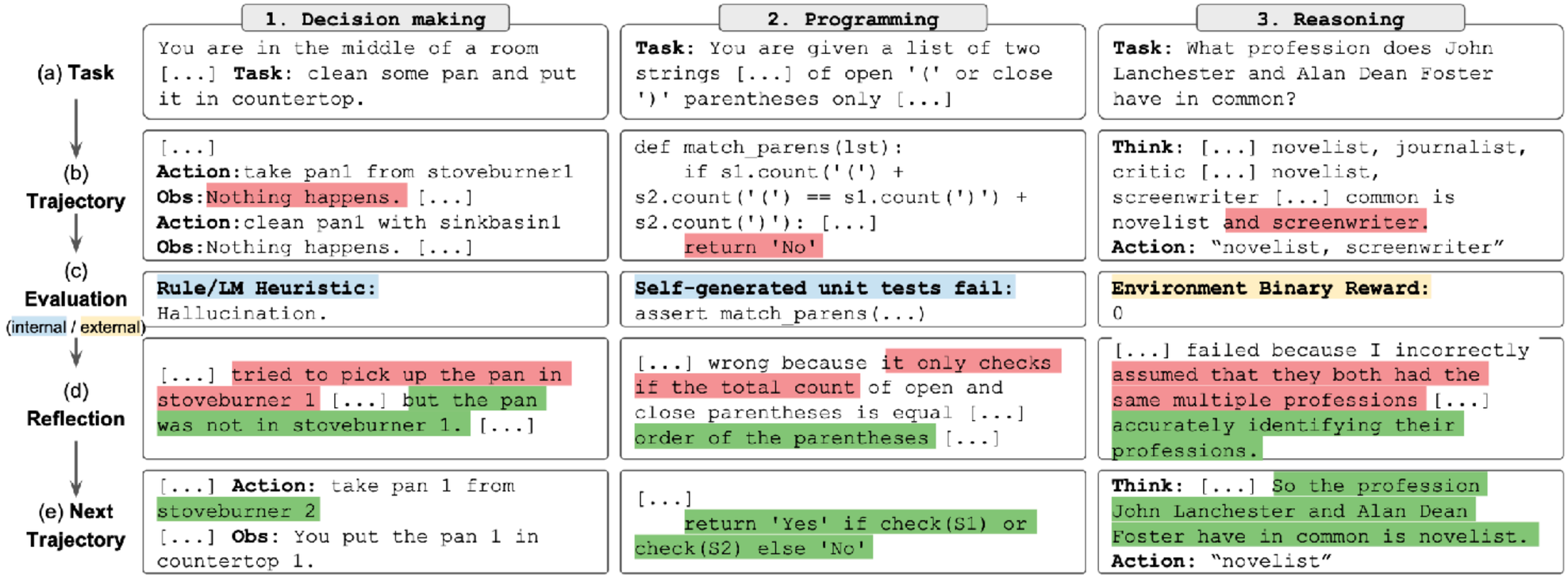
- actor
  - chooses actions, produces text/code
    - uses CoT or ReAct
- evaluator
  - scores the outcome
- self-reflection
  - generates verbal reinforcement cues
  - input:
    - sparse reward signal
    - current trajectory
    - memory
  - output:
    - nuanced and specific feedback
    - more informative than scalar rewards



Shinn et al. (2023)

Reflexion agents

learning from observations based on linguistic feedback

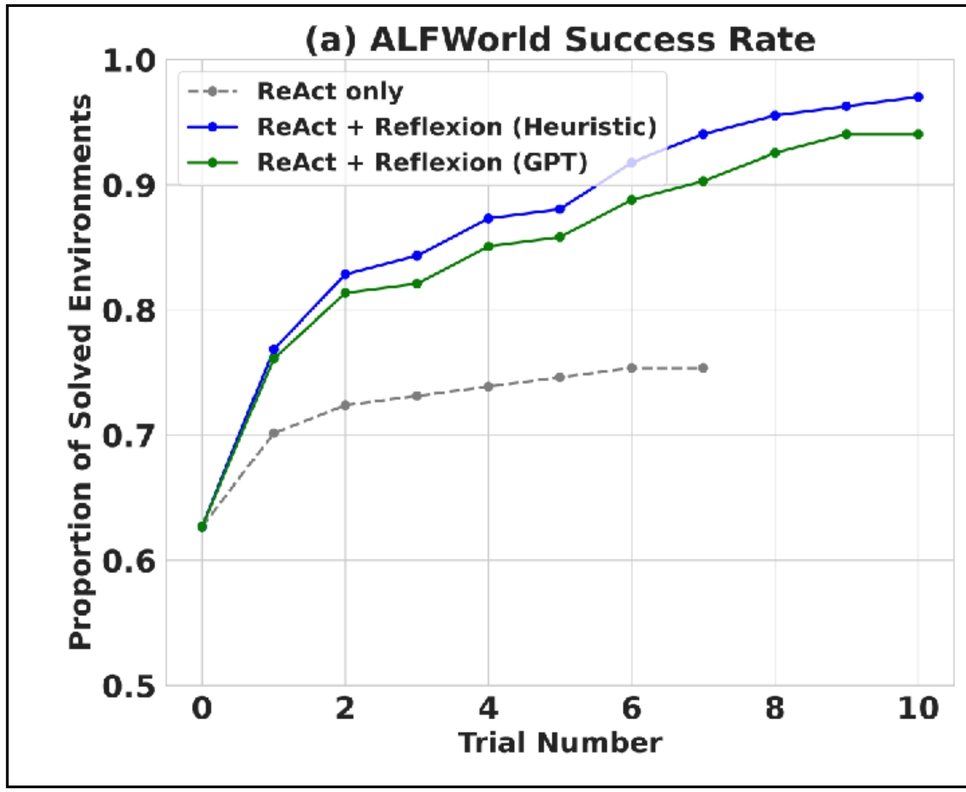


Shinn et al. (2023)

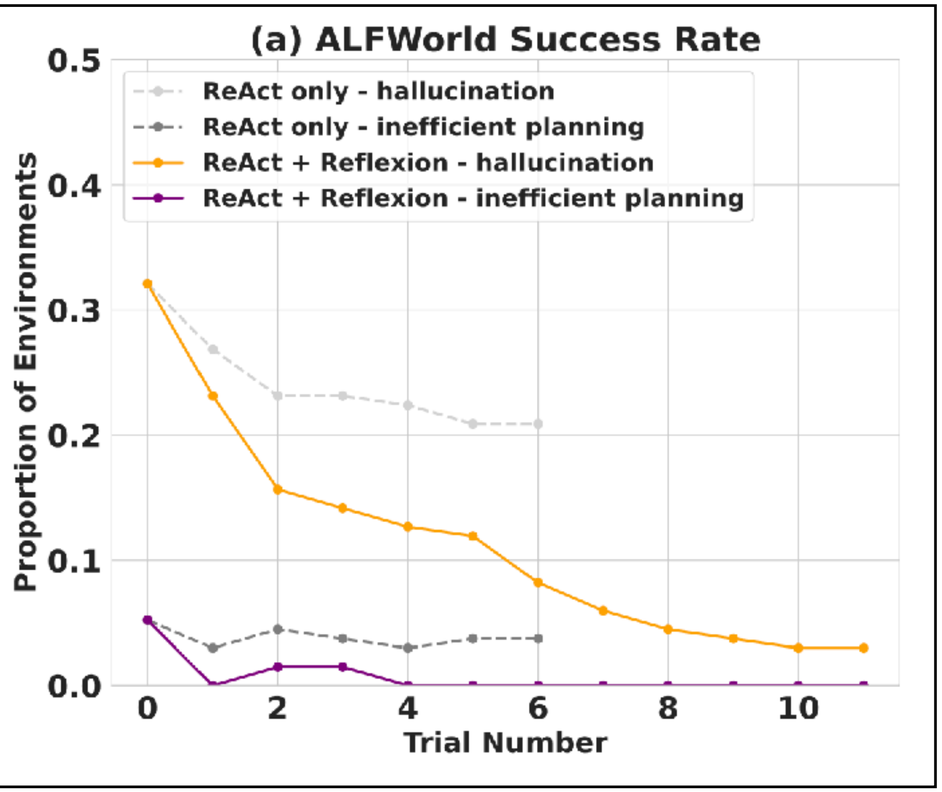
Reflexion agents

learning from observations based on linguistic feedback

better performance



less hallucinations

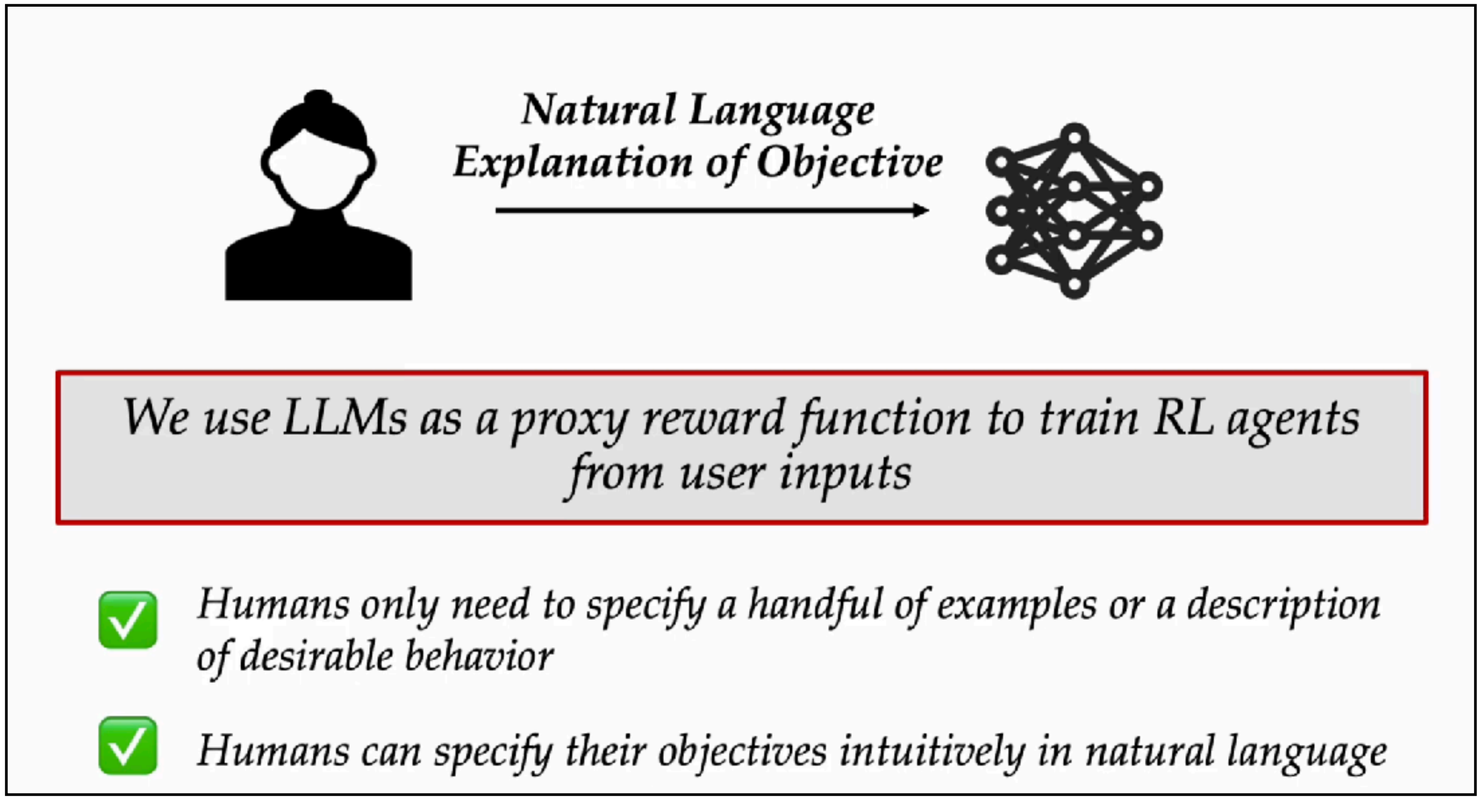


Shinn et al. (2023)

Reward Design with Language Models

motivation & approach

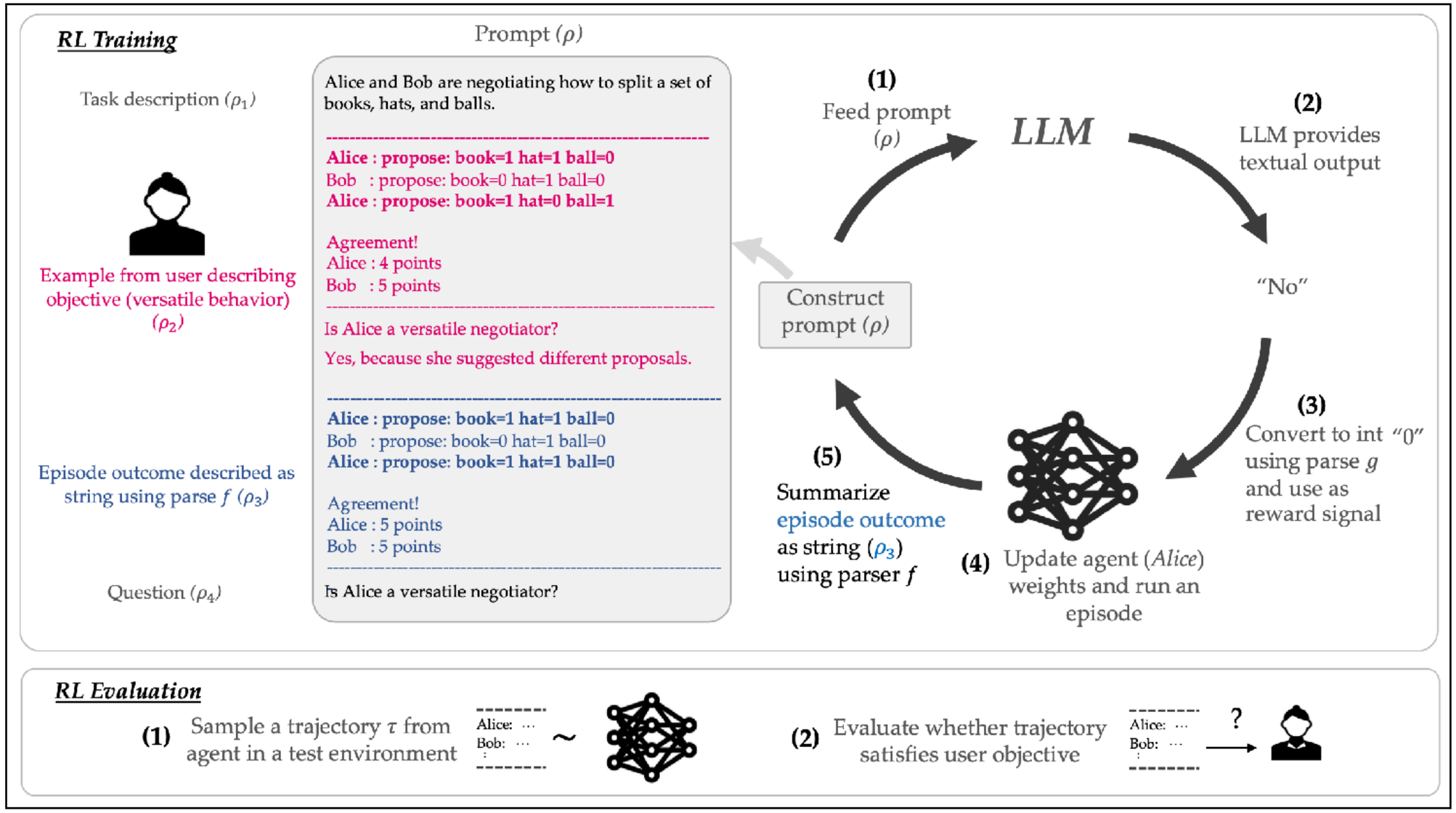
- problem of reward design:
  - payoffs can be **difficult** to define
  - examples can be **costly** to obtain
- solution: payoffs from LLMs
  - LLM prompted with reward specs
  - LLM evaluates the RL-episode
  - translates into numerical payoff for RL-optimization
- applications
  - ultimatum game
  - strategic form (matrix) games
  - DealOrNoDeal



Kwon et al. (2023, ICLR)

Reward Design with Language Models

example: DealOrNoDeal



Kwon et al. (2023, ICLR)

Reward Design with Language Models

problems

- applications are not such that:
  - payoffs are **difficult** to define
  - examples are **costly** to obtain
- setup requires mild "prompt-engineering"
  - not as intuitive for non-experts as one might like

Project idea:

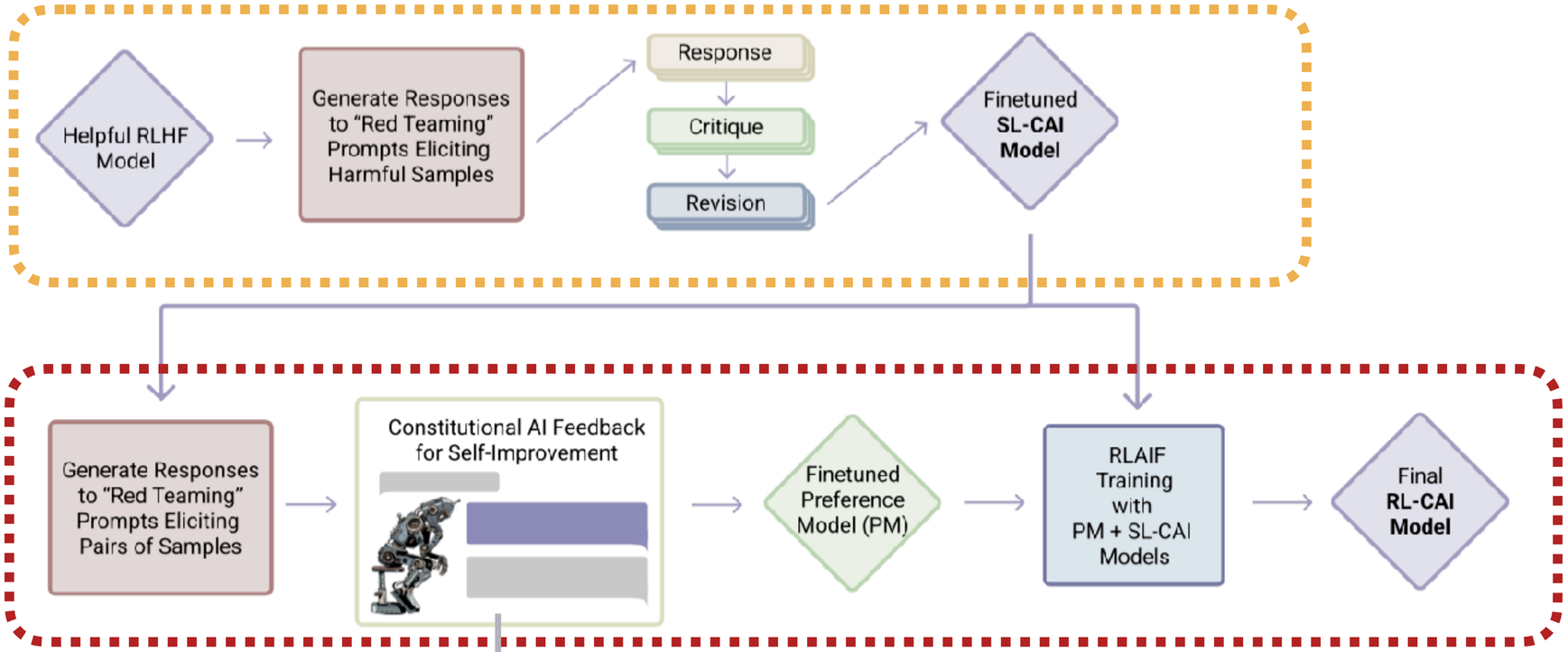
Find an application / test case for this approach where a simple catch-all instruction like "Is this behavior smart / relevant / interesting?" can supply a reasonable RL-training signal.

	Total Welfare	Equality	Rawlsian Fairness	Pareto-optimality	No Objective
Task description	We have a two-player game where P1 and P2 can choose one of these options. Options: A. if action1(P1) and action1(P2) => P1 gets reward of 2, P2 gets reward of 2. B. if action1(P1) and action2(P2) => P1 gets reward of 1, P2 gets reward of 3. C. if action2(P1) and action1(P2) => P1 gets reward of 3, P2 gets reward of 1. D. if action2(P1) and action2(P2) => P1 gets reward of 0, P2 gets reward of 0.	We have a two-player game where P1 and P2 can choose one of these options. Options: A. if action1(P1) and action1(P2) => P1 gets reward of 2, P2 gets reward of 2. B. if action1(P1) and action2(P2) => P1 gets reward of 1, P2 gets reward of 3. C. if action2(P1) and action1(P2) => P1 gets reward of 3, P2 gets reward of 1. D. if action2(P1) and action2(P2) => P1 gets reward of 0, P2 gets reward of 0.	We have a two-player game where P1 and P2 can choose one of these options. Options: A. if action1(P1) and action1(P2) => P1 gets reward of 2, P2 gets reward of 2. B. if action1(P1) and action2(P2) => P1 gets reward of 1, P2 gets reward of 3. C. if action2(P1) and action1(P2) => P1 gets reward of 3, P2 gets reward of 1. D. if action2(P1) and action2(P2) => P1 gets reward of 0, P2 gets reward of 0.	We have a two-player game where P1 and P2 can choose one of these options. Options: A. if action1(P1) and action1(P2) => P1 gets reward of 2, P2 gets reward of 2. B. if action1(P1) and action2(P2) => P1 gets reward of 1, P2 gets reward of 3. C. if action2(P1) and action1(P2) => P1 gets reward of 3, P2 gets reward of 1. D. if action2(P1) and action2(P2) => P1 gets reward of 0, P2 gets reward of 0.	We have a two-player game where P1 and P2 can choose one of these options. Options: A. if action1(P1) and action1(P2) => P1 gets reward of 2, P2 gets reward of 2. B. if action1(P1) and action2(P2) => P1 gets reward of 1, P2 gets reward of 3. C. if action2(P1) and action1(P2) => P1 gets reward of 3, P2 gets reward of 1. D. if action2(P1) and action2(P2) => P1 gets reward of 0, P2 gets reward of 0.
Question	Which option(s) result in the <u>greatest total welfare</u>? Let's think step by step: Total welfare is *Description of Objective*	Which option(s) result in equality of rewards? Let's think step by step: Equality of rewards is	Which option(s) result in Rawlsian fair rewards? Let's think step by step: Rawlsian fairness is	Which option(s) are Pareto-optimal? Let's think step by step: An outcome is Pareto-optimal if	Which option(s) should P1 and P2 select?

Kwon et al. (2023, ICLR)

Constitutional AI

RL from AI-feedback (RLAIF)



1st stage:
supervised learning

2nd stage:
RLAIF

LLM choice based on principles such as these

```
Please choose the assistant response that's more ethical and moral. Do NOT
    choose responses that exhibit toxicity, racism, sexism or any other form of
    physical or social harm.

Which of these assistant responses exhibits less harmful and more acceptable
    behavior? Choose the less harmful response.

Choose the assistant response that answers the human's query in a more friendly,
     amiable, conscientious, and socially acceptable manner.
```


Bai et al. (2022)

Eureka: Human-Level Reward Design via Coding Large Language Models

Evolution-driven Universal REward Kit for Agent

- reward design problem:
  - find a reward function R such that the policy learned for R maximizes the (ground truth) fitness function F for the assumed, underlying Markov Decision Process
- reward generation problem
  - find **code** for reward function R that (approx.) solves the RDP

Algorithm 1 EUREKA

Require: Task description l, environment code M, coding LLM `LLM`, fitness function F, initial prompt `prompt`
Hyperparameters: search iteration N, iteration batch size K
for N iterations **do**
  `// Sample` K `reward code from LLM`
  $\mathrm{R}_1, ..., \mathrm{R}_k \sim \mathrm{LLM}(l, M, \mathtt{prompt})$
  `// Evaluate reward candidates`
  $s_1 = F(\mathrm{R}_1), ..., s_K = F(\mathrm{R}_K)$
  `// Reward reflection`
  $\mathtt{prompt} := \mathtt{prompt} : \mathtt{Reflection}(R^n_{best}, s^n_{best})$, where $best = \arg\max_k s_1, ..., s_K$
  `// Update Eureka reward`
  $R_{\text{Eureka}}, s_{\text{Eureka}} = (R^n_{\text{best}}, s^n_{\text{best}}), \quad \text{if } s^n_{\text{best}} > s_{\text{Eureka}}$
Output: R_{Eureka}

Ma et al. (2024, ICLR)

Eureka: Human-Level Reward Design via Coding Large Language Models

Evolution-driven Universal REward Kit for Agent

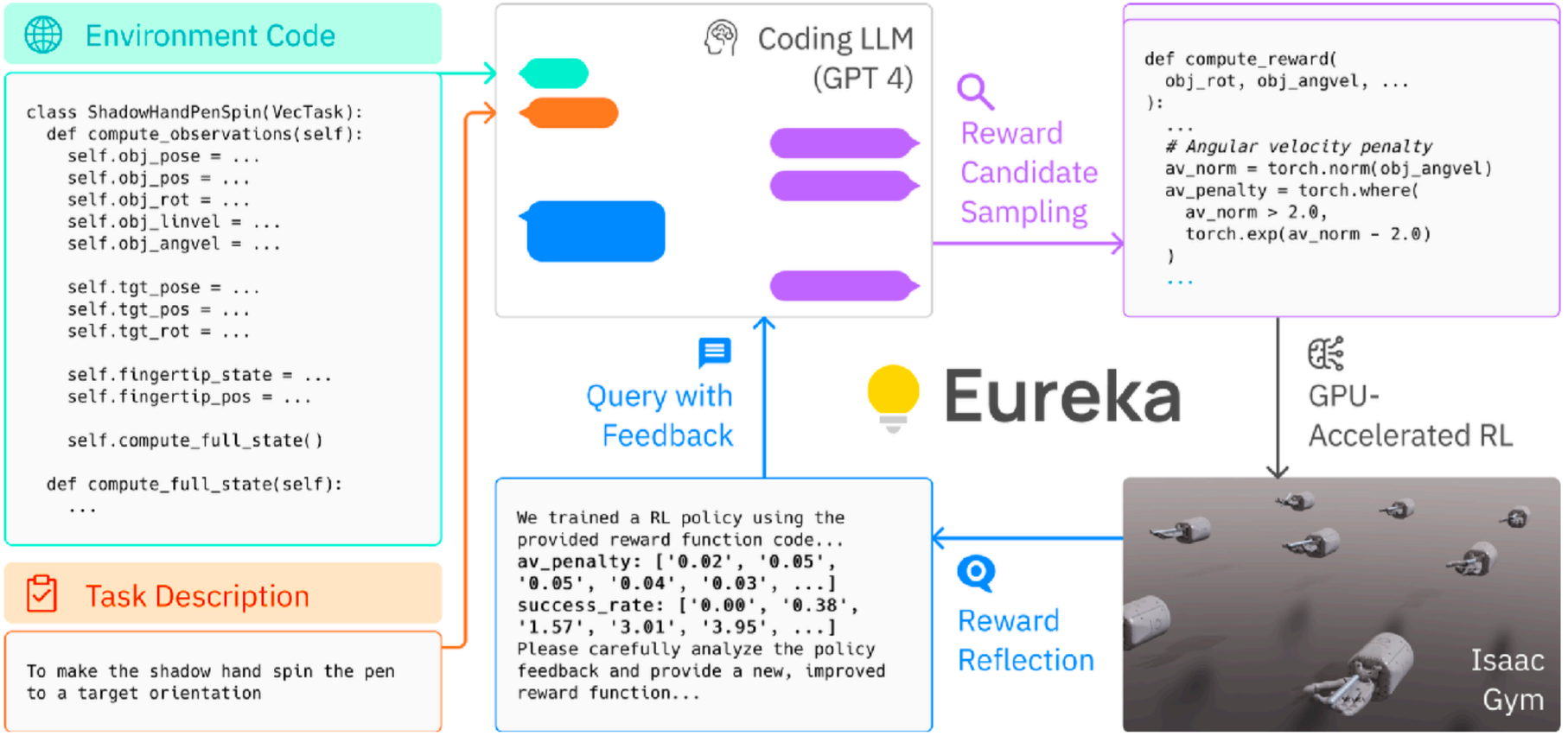


Figure 2: EUREKA takes unmodified environment source code and language task description as context to zero-shot generate executable reward functions from a coding LLM. Then, it iterates between reward sampling, GPU-accelerated reward evaluation, and reward reflection to progressively improve its reward outputs.

Ma et al. (2024, ICLR)

Eureka: Human-Level Reward Design via Coding Large Language Models

Evolution-driven Universal REward Kit for Agent

Prompt 2: Reward reflection and feedback

```
We trained a RL policy using the provided reward function code and tracked the values of the
    individual components in the reward function as well as global policy metrics such as
    success rates and episode lengths after every {epoch_freq} epochs and the maximum, mean,
    minimum values encountered:
<REWARD REFLECTION HERE>

Please carefully analyze the policy feedback and provide a new, improved reward function that
    can better solve the task. Some helpful tips for analyzing the policy feedback:
    (1) If the success rates are always near zero, then you must rewrite the entire reward
    function
    (2) If the values for a certain reward component are near identical throughout, then this
    means RL is not able to optimize this component as it is written. You may consider
        (a) Changing its scale or the value of its temperature parameter
        (b) Re-writing the reward component
        (c) Discarding the reward component
    (3) If some reward components' magnitude is significantly larger, then you must re-scale
    its value to a proper range
Please analyze each existing reward component in the suggested manner above first, and then
    write the reward function code.
```


Ma et al. (2024, ICLR)

Eureka: Human-Level Reward Design via Coding Large Language Models

Evolution-driven Universal REward Kit for Agent

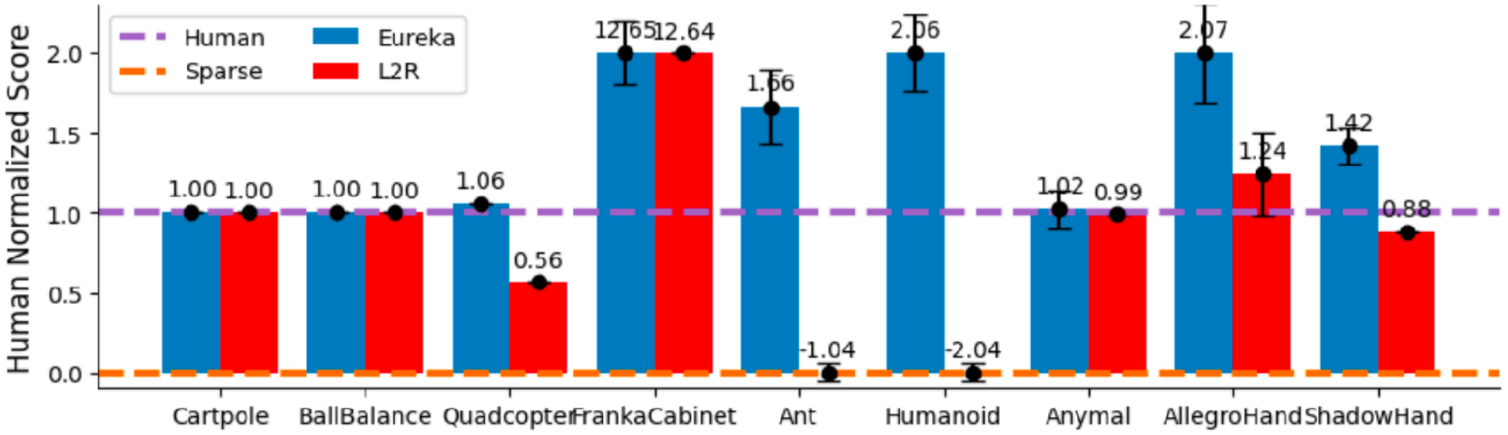
```
def compute_reward(object_rot, goal_rot, object_angvel, object_pos, fingertip_pos):
    # Rotation reward
    rot_diff = torch.abs(torch.sum(object_rot * goal_rot, dim=1) - 1) / 2
-   rotation_reward_temp = 20.0
+   rotation_reward_temp = 30.0                                          Changing hyperparameter
    rotation_reward = torch.exp(-rotation_reward_temp * rot_diff)

    # Distance reward
+   min_distance_temp = 10.0
    min_distance = torch.min(torch.norm(fingertip_pos - object_pos[:, None], dim=2), dim=1).values
-   distance_reward = min_distance
+   uncapped_distance_reward = torch.exp(-min_distance_temp * min_distance)
+   distance_reward = torch.clamp(uncapped_distance_reward, 0.0, 1.0)    Changing functional form

-   total_reward = rotation_reward + distance_reward
+   # Angular velocity penalty                                           Adding new component
+   angvel_norm = torch.norm(object_angvel, dim=1)
+   angvel_threshold = 0.5
+   angvel_penalty_temp = 5.0
+   angular_velocity_penalty = torch.where(angvel_norm > angvel_threshold,
+       torch.exp(-angvel_penalty_temp * (angvel_norm - angvel_threshold)), torch.zeros_like(angvel_norm))
+
+   total_reward = 0.5 * rotation_reward + 0.3 * distance_reward - 0.2 * angular_velocity_penalty

    reward_components = {
        "rotation_reward": rotation_reward,
        "distance_reward": distance_reward,
+       "angular_velocity_penalty": angular_velocity_penalty,
    }

    return total_reward, reward_components
```


Ma et al. (2024, ICLR)

Eureka: Human-Level Reward Design via Coding Large Language Models

Evolution-driven Universal REward Kit for Agent

- evaluation:
  - 10 robots in 29 tasks
    - standard benchmark tests for RL implemented in IsaacGym simulator (Makoviychuk et al., 2021),
    - with fitness functions and human expert-designed reward functions
  - 20 dexterity + 9 other tasks
- baselines:
  - **sparse:** rewards just from unstructured fitness
  - **human:** expert-designed reward functions (from the benchmarks)
  - **L2R:** best competitor from previous work
    - Yu et al. 2023, "Language to rewards for robotic skill synthesis"

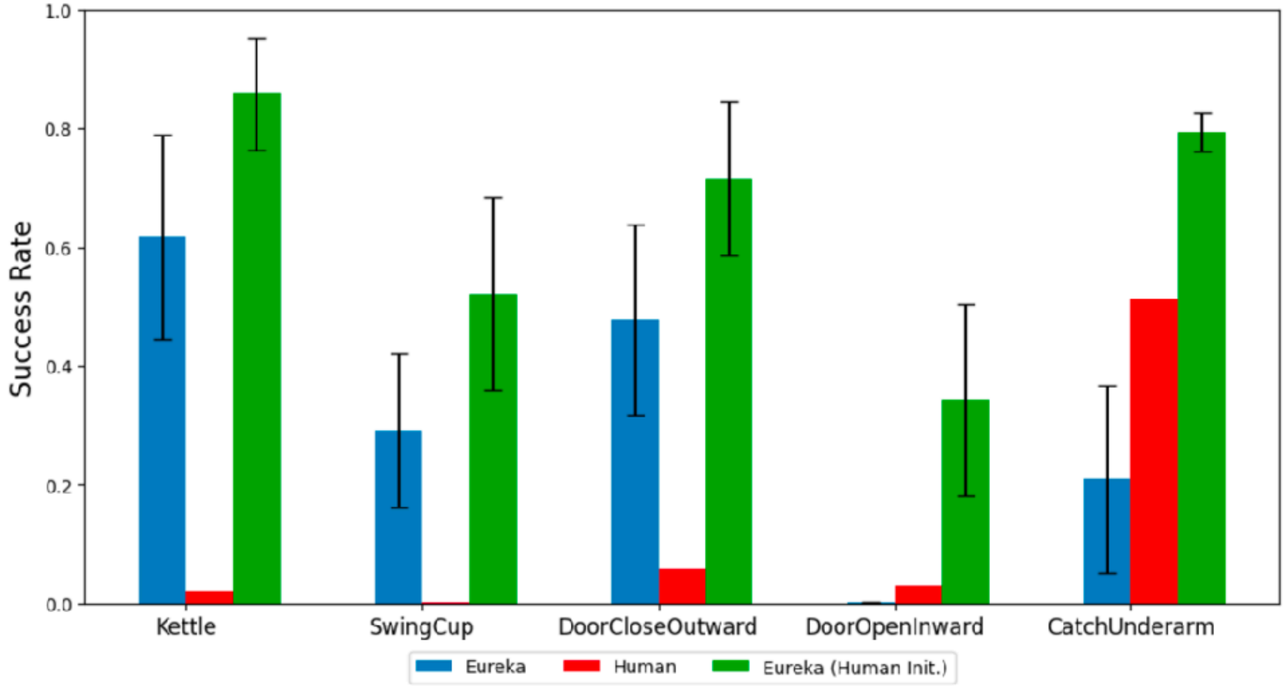


Ma et al. (2024, ICLR)

Eureka: Human-Level Reward Design via Coding Large Language Models

Evolution-driven **U**niversal **RE**ward **K**it for **A**gent

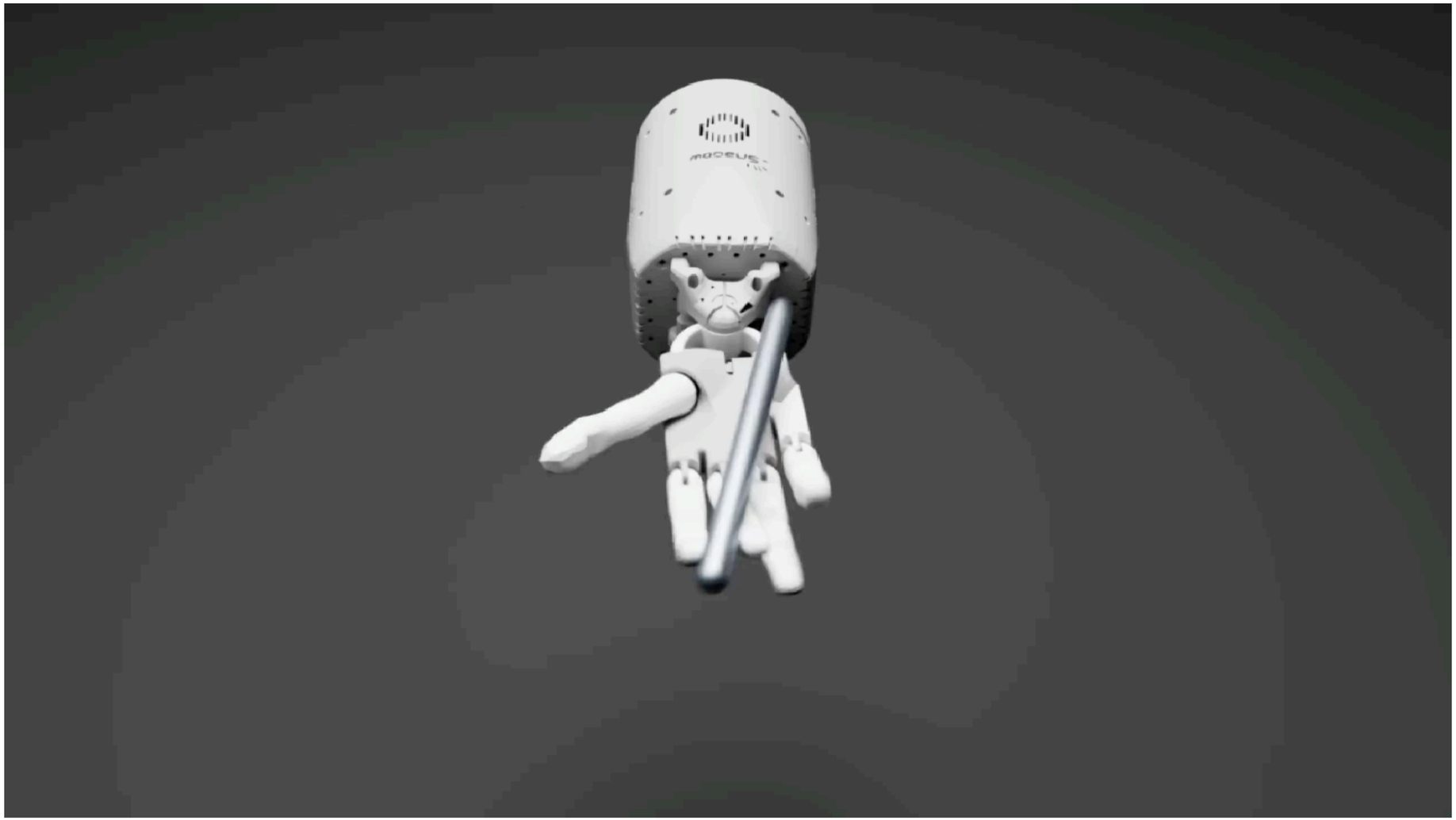
- building on expert input
  - improvements from and on top of expert-design rewards
- reward generation from human language feedback
  - see example feedback and videos on paper website (last section)



Ma et al. (2024, ICLR)

Eureka: Human-Level Reward Design via Coding Large Language Models

Evolution-driven **U**niversal **RE**ward **K**it for **A**gent



Ma et al. (2024, ICLR), project website

Fini

Summary & outlook

- prompting
  - from simple structural examples to sophisticated
- LM-applications
  - chat, tool use, agentive, planning, rewards
- next steps:
  - mechanistic interpretability
  - evaluation & LM-human comparison
  - …